

Decomposition-Based Multiobjective Evolutionary Optimization With Tabu Search for Dynamic Pickup and Delivery Problems

Junchuang Cai, Qingling Zhu, Qiuzhen Lin^{ID}, *Member, IEEE*, Zhong Ming^{ID}, and Kay Chen Tan^{ID}, *Fellow, IEEE*

Abstract—Dynamic pickup and delivery problems (DPDPs) with various constraints, such as docks, time windows, capacity, and last-in-first-out loading, have posed significant challenges for existing vehicle routing algorithms, as most of them only optimize a single weighted objective function, which makes it difficult to maintain the solutions' diversity and may easily become stuck in local optima. To alleviate this issue, this paper introduces a decomposition-based multiobjective evolutionary algorithm with tabu search for solving the above DPDPs. First, our algorithm leverages multiobjectivization and reformulates the DPDP as a multiobjective optimization problem (MOP), which is further decomposed into multiple subproblems. Then, these subproblems are approached simultaneously and collaboratively by using a crossover process to enhance the diversity of the solutions, followed by using an efficient tabu search to speed up the convergence. In this way, our algorithm can better balance the trade-off between exploration and exploitation for solving this MOP, and then one promising solution can be selected from the population to complete some pickup and delivery tasks in an interval of the DPDP. Simulation results on 64 test problems from a practical scenario of Huawei demonstrate that the proposed algorithm outperforms other competitive algorithms for tackling DPDPs. Additionally, more experiments are conducted on 20 large-scale distribution problems within JD Logistics to validate the generalization capability of our algorithm.

Index Terms—Dynamic pickup and delivery problem, multiobjectivization, decomposition, multiobjective evolutionary algorithm, tabu search, combinatorial optimization.

I. INTRODUCTION

PICKUP and delivery problems (PDPs) are an important family of vehicle routing problems (VRPs) [1], [2], [3], [4], [5], [6], [7], [8], [9], in which objects or people are transported between origin and destination [10]. PDPs belong to classical combinatorial optimization problems, which have

been proven to be NP-hard [11]. Theoretically, dynamic PDPs (DPDPs) are extended from PDPs, which further consider the realistic cases of dynamically generating orders in PDPs. In this case, vehicles are arranged at each time interval to complete some existing orders with the minimal overall cost. Most researchers focused only on the static cases of PDPs in the early stages of research [12], [13], [14], [15], [16], [17], [18], [19], [20], which rarely considered the dynamic cases of DPDPs due to their high complexity [21], [22]. As logistics applications become increasingly ubiquitous in daily life [23], studies of DPDPs have gained increasing attention in recent years. Generally, the relevant methods for addressing DPDPs can be classified into the following two categories.

The first category encompasses operational research-based optimization methods [24], in which the DPDP is decomposed into a series of PDPs, and then each PDP is solved by linear programming or mixed integer programming from scratch. In [24], an initial planning module is proposed to decompose the DPDP into a series of PDPs. Each PDP has to assign a subset of known delivery orders within a rolling-horizon framework, which is solved by a branch-and-price algorithm with various techniques, such as approximation, incomplete optimization, and a sophisticated column management scheme. These techniques are integrated to effectively balance the convergence speed and the quality of solutions in a dynamic environment. In [25], a stochastic dynamic model of DPDP is built, in which orders arrive according to a Poisson process in time. This model assumes that the pickup and delivery locations of orders are independently and uniformly distributed across the service area. Moreover, the performance bounds of the routing plans are given for the cases of using unit-capacity vehicles or multicapacity vehicles. In [26], a real-time variant of DPDP is presented to match tasks, drivers, and dedicated vehicles. In this method, a rolling horizon framework is proposed to address the matching problem, and an exact method is developed to handle new incoming information. However, the aforementioned methods require high computational costs due to the tackling of a set of static PDPs from scratch. Therefore, these approaches are unsuitable for handling rapidly changing dynamic orders in industrial scenarios.

The second category consists of various heuristic methods, which can dynamically insert new delivery orders into pre-planned routes, thereby reducing the computational costs [11]. In [27], a heuristic method is proposed for solving the rolling subproblem of DPDP, which involves the transportation of letters and small packages. This method runs the cheapest

Manuscript received 27 October 2023; revised 13 March 2024 and 26 April 2024; accepted 28 April 2024. Date of publication 22 May 2024; date of current version 4 October 2024. This work was supported in part by the National Natural Science Foundation of China under Grant 62272315, Grant 62376163, and Grant 62203308; in part by Guangdong Regional Joint Foundation Key Project under Grant 2022B1515120076; in part by the Natural Science Foundation of Guangdong Province under Grant 2023A1515011238; and in part by Shenzhen Science and Technology Program under Grant JCYJ20220531101411027. The Associate Editor for this article was X. Sun. (Corresponding authors: Qiuzhen Lin; Zhong Ming.)

Junchuang Cai, Qingling Zhu, Qiuzhen Lin, and Zhong Ming are with the College of Computer Science and Software Engineering, Shenzhen University, Shenzhen 518060, China (e-mail: qiuzhlin@szu.edu.cn; mingz@szu.edu.cn).

Kay Chen Tan is with the Department of Computing, The Hong Kong Polytechnic University, Hong Kong, SAR, China.

This article has supplementary downloadable material available at <https://doi.org/10.1109/TITS.2024.3398781>, provided by the authors.

Digital Object Identifier 10.1109/TITS.2024.3398781

insert algorithm in the early search stage and tabu search in the later search stage. In [28], a tabu search algorithm is also introduced for addressing DPDP, which uses an ejection chain-based neighborhood structure to explore new candidate solutions. In [29], a hierarchical framework with an upper-level agent and a lower-level agent is designed to tackle DPDP. A sequence of subproblems with varying scales is dynamically decomposed from the DPDP to find globally superior solutions in the upper-level agent, while multiple local search operators are applied to enhance the quality of solutions of each PDP in the lower-level agent. Moreover, to address the DPDP in a shared logistics platform, a hybrid scatter search parallel algorithm is proposed in [30], which combines a genetic algorithm, variable neighborhood search method, and tabu search. In [31], a novel variable neighborhood search algorithm, called VNSME, is designed to tackle a practical DPDP characterized by multiple constraints such as docks, time windows, capacity, and last-in-first-out (LIFO) loading. This algorithm integrates multiple local search strategies and an efficient disturbance mechanism, which has achieved the first rank in the DPDP competition at the 31st International Conference on Automated Planning and Scheduling (ICAPS 2021). Other heuristic methods, such as waiting and buffering strategies [32], [33], [34], [35], [36], order prediction algorithms [37], [38], [39], [40], variable neighborhood search [41], [42], and adaptive large neighborhood search [43], have also been employed to address DPDPs. For more details, please refer to a recent review of DPDPs [21].

The abovementioned methods have shown very promising performance for solving some specific DPDPs. However, their efficacy significantly deteriorates when facing the increased complexity of industrial DPDPs, such as the scenario presented by Huawei at the ICAPS 2021 competition, which is derived from its real-world business scenario [44]. This deterioration in performance is attributed to several factors: the dynamic nature of order arrivals, the large volume of orders, and various complex constraints including docks, time windows, capacity, and LIFO loading. Specifically, the dynamism complicates the optimization process, a large volume of orders increases the dimensionality of the solution space, and the complex constraints further restrict the feasible solution space. Although the top three algorithms¹ in the competition of ICAPS 2021 showed very promising performance, all of them only optimize a single weighted objective considering total tardiness and average traveling distance, which makes it difficult to maintain the diversity of solutions in their search process and may easily become stuck in local optima. To address this limitation, we propose a decomposition-based multiobjective evolutionary algorithm with an efficient tabu search strategy (MOEA/D-TS) for industrial DPDPs. MOEA/D-TS first reformulates the original DPDP into a multiobjective optimization problem (MOP), drawing inspiration from the efficacy of multiobjectivization in preventing convergence to local optima by reformulating a single-objective optimization problem (SOP) into an MOP [45]. To solve the above MOP, MOEA/D-TS integrates the principles of decomposition-based multiobjective

optimization and tabu search. Specifically, MOEA/D-TS further decomposes the MOP into N subproblems using diverse weight vectors, followed by restoring the solution $x_{restore}$ of the last time interval from the archive. Then, N random sequences for all new orders of the current time interval are generated, in which the cheapest insert (CI) method is applied on $x_{restore}$ to generate an initial population. In each iteration, two parent individuals are randomly selected from the population, and offspring individuals are generated through crossover operations. To iteratively evolve the offspring individuals, the tabu search method is employed, which combines four efficient local search strategies. Finally, in each time interval of DPDPs, a promising solution is selected from the population to complete some existing orders dynamically. Our algorithm can effectively balance the trade-off between exploration and exploitation in solving this DPDP, which further enhances the final optimization performance compared to the top three algorithms in the ICAPS 2021 DPDP competition and the baseline algorithm for solving this problem.

To summarize, the main contributions of this paper are the following:

(1) A decomposition-based multiobjective evolutionary algorithm (MOEA/D-TS) is proposed for solving industrial DPDPs. Initially, MOEA/D-TS reformulates the original DPDP into an MOP, adopting the strategy of multiobjectivization to prevent convergence to local optima. After decomposing the MOP into multiple subproblems, the solution's diversity and quality can be enhanced by a crossover operator and tabu search, which yields the best performance for solving the industrial DPDP, as verified in our experimental studies.

(2) A novel crossover operator is designed to generate offspring with high diversity from various parent solutions, which avoids premature convergence to local optima in the complex landscape of industrial DPDPs. First, each offspring inherits routes from randomly selected parents. Then, duplicate nodes are removed to ensure feasibility, and any unassigned orders are incorporated into the planned routes to form a comprehensive solution.

(3) A tabu search method is designed in MOEA/D-TS to run on the above high-potential solutions. This tabu search method combines four efficient local search strategies, such as couple-exchange, block-exchange, couple-relocate, and block-relocate, to further evolve the offspring solutions. In this way, the quality of solutions is further improved.

A number of empirical studies were conducted to address 64 industrial benchmark problems introduced by Huawei in the ICAPS 2021 DPDP competition [44], which validate the superior performance of MOEA/D-TS in terms of solution quality when compared to five recently proposed algorithms (i.e., the top three and baseline algorithms of the competition, and HMTS [46]). Moreover, a series of comparative experiments were conducted on 20 real-world benchmark problems derived from the logistics distribution system of Jingdong [13], which demonstrate the generalization capability of MOEA/D-TS, as MOEA/D-TS consistently outperforms five recently proposed algorithms (MATE [13], EMA [1], CCMO [47], VNSME [31], and MBEA [48]).

¹<https://competition.huaweicloud.com/information/1000041411/Winning>

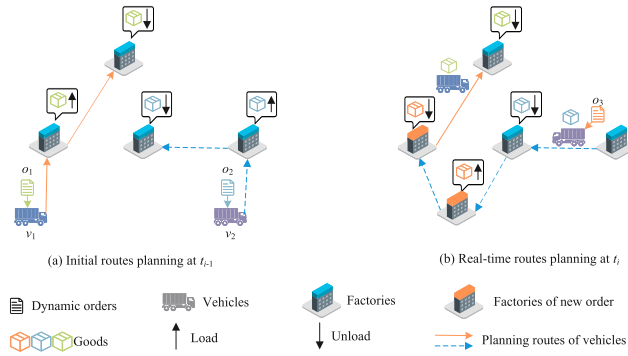


Fig. 1. A toy example of DPDP.

The rest of this paper is organized as follows. Section II presents some background information of this paper and our motivation to design the proposed MOEA/D-TS. Then, Section III introduces the proposed MOEA/D-TS. Experimental results are provided and analyzed in Section IV. Finally, Section V presents the conclusion and outlines the future directions of this research.

II. BACKGROUND AND MOTIVATION

In Section II-A, the mathematical formulation of the industrial DPDP is presented, followed by the introduction of a state-of-the-art multiobjective evolutionary algorithm (MOEA/D [49]) in Section II-B. Furthermore, a brief review of tabu search is presented in Section II-C, while the motivation to design our MOEA/D-TS is clarified in Section II-D.

A. Mathematical Formulation for Common Industrial DPDPs

In this paper, a common industrial DPDP is investigated, in which orders are dynamically generated in real-time at different factories throughout a day. The target of this DPDP is to minimize the total costs by efficiently transporting goods from pickup to delivery nodes. In this DPDP, the logistics scenario is simulated using a simulator to divide a day into a number of fixed time intervals, so that new orders are expected to be completed dynamically in the nearest time interval. As shown in Fig. 1, the planning outcomes of each time interval are interdependent, and the current planning results will influence both the current vehicles' remaining capacity and their relative positions to complete subsequent orders. Additionally, after the planning outcomes in each time interval, some existing orders are completed dynamically.

The input of the industrial DPDP comprises the following components: M nodes (factories) $\{F_i | i = 0, 1, 2, \dots, M\}$, where F is a set of M nodes. Each factory is equipped with L_d docks for vehicles to load or unload goods. A complete directed graph route network $G = (F, A)$ is defined, where the edge set $A = \{< i, j > | i, j \in M, i \neq j\}$, and each edge $< i, j >$ is associated with a nonnegative actual distance D_{ij} . Moreover, a set of orders O with a quantity of C is included. Each order $o_i = \{F_p^i, F_d^i, Q^i, t_e^i, t_l^i\}$ can be dynamically generated by different factories at any time within a day. Here, F_p^i and F_d^i represent the pickup and delivery nodes,

respectively. Q^i denotes the quantity of goods in order o_i . The time at which the order is created is represented as t_e^i , whereas t_l^i signifies the committed completion time. The fleet consists of homogeneous vehicles, $V = \{v_k | k = 1, 2, \dots, K\}$, where each vehicle v_k has a fixed capacity Q , and K represents the total number of vehicles. The initial position for each vehicle is randomly located at a factory.

The output of this industrial DPDP includes the route planning $RP = \{rp_k = \{n_1^k, n_2^k, \dots, n_{l_k}^k\} | k = 1, 2, \dots, K\}$, where n_i^k and l_k indicate the i -th factory and the total number of factories visited by vehicle v_k , respectively. Two optimization objectives are considered here. The primary objective, denoted as f_1 , represents the sum of the total tardiness for all orders served by vehicles. It is expressed as follows:

$$f_1 = \sum_{k \in V} \sum_{i \in l_k} \max(0, t_i^k - t_l^i) d_i^k. \quad (1)$$

Here, t_i^k represents the arrival time of vehicle v_k at the i -th node. The order corresponding to the i -th node visited by vehicle v_k is denoted as y_i^k , and t_l^i signifies the committed completion time for order y_i^k . The binary variable d_i^k represents the type of the i -th node where vehicle v_k arrives, with a value of 1 (or 0) indicating a delivery (or pickup) node. Notably, the tardiness of orders is calculated and accumulated only when vehicle v_k arrives at its delivery nodes. Furthermore, the arrival time t_{i+1}^k of vehicle v_k at the $(i+1)$ -th node can be determined using the following equation:

$$t_{i+1}^k = t_i^k + w_i^k + T_{da} + t_S^{y_i^k} + t_{i,i+1}^k, \forall i \in l_k, \forall k \in V, \quad (2)$$

where w_i^k denotes the queuing time for an available dock to load or unload goods at node i after vehicle v_k arrives, T_{da} corresponds to the docking time at the dock from the factory for vehicle v_k , $t_S^{y_i^k}$ represents the time spent by vehicle v_k for loading or unloading goods of order y_i^k at the i -th node, and $t_{i,i+1}^k$ indicates the travel time for vehicle v_k to travel from the i -th node to the $(i+1)$ -th node after completing the service at the i -th node.

The second objective f_2 aims to minimize the average travel distance of vehicles, as defined by

$$f_2 = \frac{1}{K} \sum_{k \in V} \sum_{i \in l_k} D_{n_i^k, n_{i+1}^k}. \quad (3)$$

where $D_{n_i^k, n_{i+1}^k}$ denotes the distance from node n_i^k to node n_{i+1}^k . The above two objectives are used to guide the population search, aiming to enhance the diversity of solutions.

The objective functions f_1 and f_2 inherently conflict. Consider a scenario where numerous orders share identical pickup and delivery addresses but are generated at different times. Servicing each order immediately upon generation minimizes f_1 at the expense of maximizing f_2 . Conversely, delaying earlier orders to aggregate those with matching pickup and delivery addresses for collective vehicle servicing results in f_1 attaining its maximum value and f_2 reaching its minimum.

In each time interval, one route planning solution should be output to complete some existing orders. In this case,

the overall objective is defined in Eq. (4) to select the route planning solution. Here, α is a positive constant denoting the penalty for each second of order tardiness, emphasizing the importance of punctual order delivery in the real-world to improve customer satisfaction.

$$TC = \alpha \times f_1 + f_2, \quad (4)$$

$$s.t. \ q_{i+1}^k - q_i^k - Q^{y_{i+1}} = 0, \quad \forall i, i+1 \in l_k, \quad \forall k \in V \quad (5)$$

$$0 \leq q_i^k \leq Q, \quad \forall i \in l_k, \quad \forall k \in V \quad (6)$$

$$t_e^i \leq t_i^k \leq t_l^i, \quad \forall i \in O \quad (7)$$

$$0 \leq t_{F_p}^k \leq t_{F_d}^k, \quad \forall i \in O, \quad \forall k \in V, \quad \forall F_p^i, F_d^i \in F \quad (8)$$

$$0 \leq \sum_{k \in V} z_i^k \leq L_d, \quad \forall i \in F \quad (9)$$

$$x_{F_p^i, F_p^j}^k - x_{F_d^i, F_d^j}^k = 0, \quad \forall i, j \in O, \quad \forall F_p^i, F_p^j, F_d^i, F_d^j \in F \quad (10)$$

$$\sum_{k \in V} \sum_{i \in l_k} y_i^k d_i^k = O. \quad (11)$$

Eqs. (5) and (6) establish constraints concerning the vehicle capacity. Eq. (7) gives the pickup and delivery time window for each order, wherein the right-hand side inequality constitutes a soft constraint. A penalty, represented by f_1 , is applied if an order is delivered after the committed completion time. To maintain operational sequencing, Eq. (8) guarantees that vehicles first visit the pickup node before advancing to the associated delivery node. Furthermore, Eq. (9) ensures that the simultaneous occupancy of vehicles at a factory's dock does not surpass the available dock count. The LIFO loading constraint is articulated by Eq. (10), while Eq. (11) confirms that every order receives vehicle assignment for delivery.

B. The Framework of MOEA/D

A decomposition-based multiobjective evolutionary algorithm (MOEA/D) [49] has received significant attention [50], [51], [52], which is employed in this paper. It transforms an MOP into a collection of single-objective optimization problems by the Tchebycheff method, as expressed by:

$$g(x|\lambda, z) = \min_{1 \leq i \leq m} [\max_{1 \leq i \leq m} \lambda_i |f_i(x) - z_i|] \quad (12)$$

s.t. $x \in \Omega$.

Here, $\lambda = \{\lambda_1, \dots, \lambda_m\}$ represents a weight vector, where $\sum_{i=1}^m \lambda_i = 1$ for all $i = 1, \dots, m$, and $\lambda_i \geq 0$. m denotes the number of objectives, while Ω stands for the decision space. In addition, $f_i(x)$ represents the i -th objective function, while $z = \{z_1, \dots, z_m\}$ signifies a reference point capturing the minimum of each objective function.

To provide an explicit introduction to MOEA/D, we have outlined its pseudocode in Algorithm 1, incorporating five inputs: the count of subproblems (N), a set of N weight vectors ($\lambda = \{\lambda_1, \dots, \lambda_m\}$), the quantity of weight vectors in the neighborhood (T), the likelihood of selecting parent solutions from the neighborhood (δ), and the maximum count of parent solutions replaced by each offspring (nr). Finally, a solution set ($x^1, \dots, x^N$) is generated from MOEA/D to

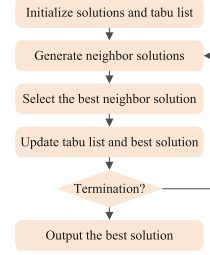


Fig. 2. The process of tabu search.

approximate the Pareto-optimal set (PS). Here, Algorithm 1 includes four distinct modules:

(1) Initialization (lines 2-8): The algorithm begins by calculating the Euclidean distances between each pair of weight vectors. Subsequently, it determines and records the T nearest weight vectors to each vector in set B (lines 3-5). Next, it generates an initial population randomly across the solution space, and then constructs the reference point z of the population.

(2) Evolution (lines 12-14): A random number $rand$ within the range $[0,1]$ is generated for each subproblem $i = 1, 2, \dots, N$. Here, set

$$P = \begin{cases} B(i) & \text{if } rand < \delta \\ \{1, \dots, N\} & \text{otherwise.} \end{cases} \quad (13)$$

Subsequently, two indexes r_1 and r_2 are randomly selected from P (line 13). The offspring y are then generated through the integration of differential evolution (DE) and polynomial mutation (PM), utilizing x^i , x^{r_1} , and x^{r_2} as inputs (line 14).

(3) Update of reference point (line 16): For each $j = 1, \dots, m$, if z_j is greater than $f_j(y)$, it is assigned as $f_j(y)$.

(4) Update of population (lines 18-25): The solution replacement count (c) is initially set to 0, and then an index j is randomly chosen from P . Subsequently, solution x^j is updated by offspring y , and c is increased by 1 if aggregated function $g(y|\lambda^j, z)$ is less than $g(x^j|\lambda^j, z)$. Following this, j is eliminated from P . Notably, c cannot exceed nr .

C. A Brief Review of Tabu Search

Tabu search was first introduced in [53], which is a prominent metaheuristic algorithm for tackling diverse combinatorial optimization problems. As presented in Fig. 2, the foundational components of tabu search [54] encompass a set of moves for generating neighboring solutions and a tabu list that constrains specific moves. One solution is designated as both the current and the best solution, which is also logged into the tabu list. Then, tabu search iterates the following process until the termination condition is satisfied. First, various neighboring solutions are generated by applying a set of moves to the current solution. Second, each of these generated solutions is assessed to determine its status, either as tabu or allowable, using the tabu list as a reference. A solution that matches any entry in the tabu list is deemed tabu, while those that do not match any entry are marked as allowable. The subsequent solution is determined from the best-performing allowable neighbors. If this selected solution outperforms the current best solution, it becomes the newly best solution, which is then added to the tabu list. If the list surpasses its predetermined capacity, the oldest solution is removed from the list.

Algorithm 1 MOEA/D ($N, \lambda, T, \delta, n_r$)

```

1: /* Initialization */
2: /*  $B = \text{Initialize-Weight-Vector}(N, \lambda, T)$  */
3: for  $i := 1$  to  $N$  do
4:   Obtain  $\lambda^{i_1}, \dots, \lambda^{i_T}$  (the  $T$  nearest weight vectors of
      $\lambda^i$ ) and define  $B(i) = \{i_1, \dots, i_T\}$ 
5: end for
6: Initialize a population  $x^1, \dots, x^N$  and evaluate its objec-
   tives;
7: /*  $z = \text{Initialize-Reference-Point}(x)$  */
8: Initialize  $z = \{z_1, \dots, z_m\}$  with  $\min_{1 \leq i \leq N} f_j(x^i)$ ;
9: while The stopping criterion is not satisfied do
10:   for  $i := 1$  to  $N$  do
11:     /* Evolution */
12:      $P = \text{rand} < \delta ? B(i) : 1, \dots, N$ ;
13:     Randomly pick two indexes  $r_1$  and  $r_2$  from  $P$ ;
14:     Obtain  $y$  by applying DE and PM on solutions
        $x^i, x^{r_1}, x^{r_2}$ ;
15:     /*  $z = \text{Update-Reference-Point}(y, z)$  */
16:     For each  $j = 1, \dots, m$ , if  $z_j > f_j(y)$ , then update
        $z_j = f_j(y)$ ;
17:     /*  $x = \text{Update-Population}(P, y, x)$  */
18:     Set  $c = 0$ ;
19:     while  $c < n_r$  and  $P \neq \emptyset$  do
20:       Randomly pickup an index  $j$  from  $P$ ;
21:       if  $g(y|\lambda^j, z) < g(x^j|\lambda^j, z)$  then
22:          $x^j \leftarrow y$ , and  $c \leftarrow c + 1$ ;
23:       end if
24:       Delete index  $j$  from  $P$ ;
25:     end while
26:   end for
27: end while
28: Output: a solution set  $(x^1, \dots, x^N)$  to approximate the
   PS;

```

The tabu list plays a pivotal role in averting solution cycling, thereby preventing the repeated exploration of the same points in the search space. This mechanism steers the search toward diverse directions and avoids becoming trapped in local optima.

D. Motivation

In evolutionary computation, the process of transforming an SOP into an MOP is termed “multiobjectivization”, which enables the application of multiobjective optimization methods to identify the optimal solution for the original SOP [55], [56]. One prevalent approach for achieving multiobjectivization involves decomposing the original objective into multiple subobjectives [57], [58], which are then solved simultaneously to find the optimal solution of the original SOP. By addressing the newly formed MOP instead of the original SOP, multiobjectivization can yield an improved solution for the original SOP, as it has the potential to improve the solutions’ diversity [59], [60] and escape from local optima [45], [61]. Nonetheless, an industrial DPDP, inherently as an SOP with intricate constraints such as docks, time windows, capacity, and LIFO loading, has posed significant challenges to existing

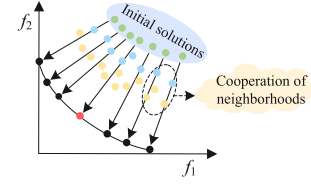


Fig. 3. Simple illustration of using multiobjectivization to optimize the original DPDP, where the solution marked with red color is selected to complete some existing orders.

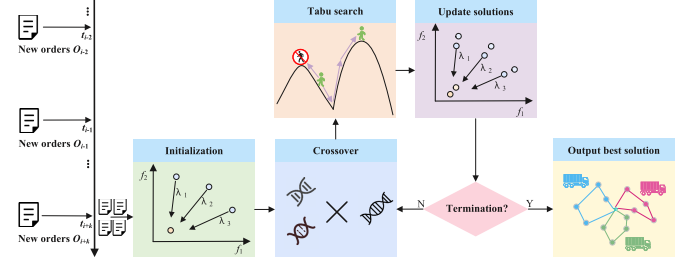


Fig. 4. The framework of the proposed algorithm.

vehicle routing algorithms, as most of them only optimize a single weighted objective function in their search process, which are difficult to maintain the solutions’ diversity and may easily become trapped in local optima. Therefore, the formulation of the original DPDP into an MOP during optimization may enhance the solutions’ diversity and lead to an improved optimization performance.

Based on the above insights, our algorithm first reformulates the original DPDP into an MOP with two subobjectives (f_1 and f_2) by using multiobjectivization, which is further decomposed into multiple subproblems using a set of weight vectors $\lambda^1, \lambda^2, \dots, \lambda^N$. These subproblems are then approached by using a crossover process to enhance the diversity of solutions, followed by using an efficient tabu search to speed up the convergence. As illustrated in Fig. 3, at each time interval, an MOP is first reformulated by adopting multiobjectivization. To effectively solve the MOP, different subproblems collaborate with each other throughout the optimization process, culminating in the selection of an optimal solution (highlighted by the red point) with the minimum TC value to complete some existing orders for the original DPDP. In this way, MOEA/D-TS can enhance the diversity of solutions and avoid becoming trapped in local optima, leading to the finding of a more promising solution for the original problem.

III. PROPOSED ALGORITHM

In this section, we first present the algorithmic framework of MOEA/D-TS and then provide a discussion of its two primary components: the crossover process and tabu search process. Fig. 4 illustrates the flowchart of MOEA/D-TS. After initialization, MOEA/D-TS iteratively runs crossover and tabu search until the termination condition is met. These components ensure that all generated solutions consistently meet constraints related to time windows, capacity, docks, and LIFO loading.

A. Overall Framework

In industrial DPDPs, each day is usually divided into a number of fixed time intervals, so that the coming orders are expected to be completed in one nearest time interval. During each interval, as new orders emerge, MOEA/D-TS is used to derive candidate solutions, addressing both order assignment and vehicle routing. The framework of MOEA/D-TS is outlined in Algorithm 2. Its initialization consists of three pivotal steps (lines 2-6): generating the T nearest weight vectors for each subproblem, creating a population $x^1, \dots, x^N$, and establishing a reference point z . The method for generating the T nearest weight vectors aligns with the procedure delineated in MOEA/D [49], which ensures the production of uniformly distributed weight vectors and the selection of the T weight vectors closest to λ^i for each subproblem i , thereby forming their neighborhoods (line 2). To initialize the population, the solution from the previous interval, denoted as $x_{restore}$, is restored from the archive. Then, N random permutation sequences are created by incorporating all new orders in the current interval. Based on these sequences, the CI method is applied to $x_{restore}$ to generate an initial population (line 5). Any solution that violates the DPDP constraints (e.g., capacity, docks, and LIFO loading) is immediately discarded during initialization. After that, MOEA/D-TS undertakes a cycle of processes (lines 10-13): crossover, tabu search, reference point update, and population update. The algorithm terminates when either the maximum number of iterations or the maximum runtime is reached. The subsequent sections provide in-depth explanations of the crossover and tabu search processes.

B. Crossover

Our novel crossover operator is specifically designed to enhance offspring's diversity, which is a critical factor in avoiding local optima in the rugged landscape of industrial DPDPs. The rationale behind its design lies in its unique ability to combine routes from diverse parents, thus fostering a broader exploration of the solution space.

This simple yet effective crossover operator is tailored for industrial DPDPs, as outlined in Algorithm 3. This operator works by sequentially selecting a random route from the parent solutions as the offspring, denoted as x_{child} . To ensure feasibility, all duplicate nodes are eliminated. This procedure continues until all vehicle routes are integrated into x_{child} (lines 2-6). Thereafter, to formulate a comprehensive solution, any remaining unassigned orders are incorporated into x_{child} . For this purpose, these unassigned orders are grouped into a set U (line 7). Each order in U , represented by o_i , is assigned to x_{child} at the position corresponding to the minimum value of (12) (lines 8-10). Finally, the crossover operator can yield a complete solution in x_{child} . Considering the intricacies of industrial DPDP, traditional methods may easily become trapped in local optima. To overcome this challenge, our crossover operator is engineered to generate offspring with increased diversity, diminishing the propensity toward local optima. This strategy subsequently empowers the tabu search to exploit the global optimum more effectively. Based on Algorithm 3, it can be inferred that the worst-case time complexity of the crossover operator is $O(K + |U|)$, where

Algorithm 2 General Structure of MOEA/D-TS

```

1: Input: A DPDP instance;
2:  $B = \text{Initialize-Weight-Vector}(N, \lambda, T)$ ; //function in
   Algorithm 1
3: /* Initialize population */
4: Restore the solution  $x_{restore}$  of the last time interval from
   the archive;
5: For  $N$  random sequences of new orders, apply the
   CI method on  $x_{restore}$  to obtain an initial population
    $x^1, \dots, x^N$ ;
6:  $z = \text{Initialize-Reference-Point}(x)$ ; //function in
   Algorithm 1
7: while Stopping criterion is not satisfied do
8:   for  $i := 1$  to  $N$  do
9:     Randomly choose two parents  $p_1, p_2$  from  $P$ ;
10:     $x_{child} \leftarrow \text{Crossover}(p_1, p_2, K)$ ; //Algorithm 3
11:     $x_{child} \leftarrow \text{Tabu-Search}(x_{child})$ ; //Algorithm 4
12:     $z = \text{Update-Reference-Point}(x_{child}, z)$ ; //function in
       Algorithm 1
13:     $x = \text{Update-Population}(P, x_{child}, x)$ ; //function in
       Algorithm 1
14:   end for
15: end while
16:  $x^* \leftarrow \text{Select\_Best}(x)$ ; //select the best solution according
   to  $TC$ 
17: Archive the best solution  $x^*$  of the current time interval;
18: Output: The best solution  $x^*$  of the current time interval.
```

Algorithm 3 Crossover(p_1, p_2, K)

```

1: Input: parent solutions  $p_1, p_2$ , the number of all vehicles:
    $K$ ;
2:  $x_{child} \leftarrow \emptyset$ ;
3: for  $k := 1$  to  $K$  do
4:   Randomly copy the route  $r_k$  from  $p_1, p_2$  to solution
    $x_{child}$ ;
5:   Repair  $x_{child}$  by deleting the duplicate nodes;
6: end for
7:  $U \leftarrow$  all the remaining unassigned orders;
8: for all  $o_i \in U$  do
9:   Insert  $o_i$  to  $x_{child}$  with the minimum value of (12);
10: end for
11: Output: the offspring solution:  $x_{child}$ .
```

K denotes the total number of vehicles and $|U|$ represents the count of unassigned orders.

C. Tabu Search

The tabu search method, employing a combination of four advanced local search operators (couple-exchange, block-exchange, couple-relocate, and block-relocate), is tailored to the complex landscape of industrial DPDPs, where conventional methods fall short in navigating the intricate solution space effectively. These four operators are selected for their dual merits (efficiency and precision), which are suitable for optimizing the DPDP's inherent complexity of specific pickup and delivery nodes for each order. Furthermore, this

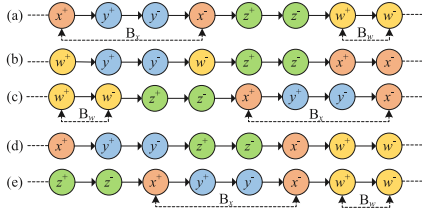


Fig. 5. (a) The initial route. (b) The new route created by couple-exchange. (c) The new route created by block-exchange. (d) The new route created by couple-relocate. (e) The new route created by block-relocate.

method's adaptability to high-potential solutions, derived from the initial crossover process, ensures continuous improvement in solution's quality, distinguishing our approach from existing techniques.

To elucidate the steps involved in the tabu search process, we outline its pseudocode in Algorithm 4, using the offspring solution x_{child} resulting from the crossover operator as the input. In Algorithm 4, a tabu search is utilized to refine the offspring solution x_{child} . Here, one of the four local search operators is randomly chosen to generate a neighboring solution, as indicated in line 7. Detailed descriptions of these operators are available in [62]. For instance, the couple-exchange operator interchanges two couples (e.g., $[x^+, x^-]$ and $[w^+, w^-]$) to derive a new route (Fig. 5(b)). The block-exchange operator interchanges two blocks (e.g., B_x and B_w) to yield a new route (Fig. 5(c)). The couple-relocate operator shifts a couple (e.g., z^+, z^-) to a different feasible location, resulting in a distinct route (Fig. 5(d)). Similarly, the block-relocate operator moves a block (e.g., B_x) to a new feasible location, forming another route (Fig. 5(e)). The local search on the current solution $x_{current}$ is run for specific iterations, represented by $NeighborThreshold$. During each iteration, one local search operator is chosen at random. The best neighbor solution $x_{bestNeighbor}$ is then selected from the generated neighbors, evaluating both its optimality (i.e., the minimum value of TC in Eq. (4)) and whether it is absent from the tabu list, as outlined in lines 5-11. Additionally, $x_{bestNeighbor}$ is compared with the current best solution x_{best} . If it demonstrates a reduced total cost, x_{best} is updated accordingly, as depicted in lines 12-14. Simultaneously, the current solution is replaced by the best neighbor solution, and the tabu list is updated, as highlighted in lines 15-16. After iteratively applying the aforementioned operations to the offspring solution x_{child} for a maximum number of iterations ($MaxIter$), an improved solution x_{best} with the minimal cost is selected for subsequent evolution. Moreover, the worst-case time complexity of the tabu search operation is $O(MaxIter \times NeighborThreshold)$, which can be readily derived from Algorithm 4.

IV. EXPERIMENTAL STUDY

A. Benchmark Problems and Performance Metrics

In this study, a comprehensive DPDP benchmark set (the HW problems [44]) is used to assess the efficacy of MOEA/D-TS. These benchmarks, introduced by Huawei during the DPDP competition of ICAPS 2021, are developed from

Algorithm 4 Tabu-Search(x_{child})

```

1: Input: An offspring solution  $x_{child}$ ;
2:  $x_{best} = x_{child}, x_{current} = x_{child}$ ;
3: while  $iter < MaxIter$  do
4:    $x_{bestNeighbor} = x_{current}$ ;
5:   while  $nIter < NeighborThreshold$  do
6:     /* Generate a neighbor solution */
7:      $x_{tmp} \leftarrow$  Run a randomly selected local search on
        $x_{current}$ ;
8:     if  $x_{tmp}$  is non-tabu and  $TC(x_{tmp}) <$ 
        $TC(x_{bestNeighbor})$  then
9:        $x_{bestNeighbor} \leftarrow x_{tmp}$ ;
10:    end if
11:  end while
12:  if  $TC(x_{bestNeighbor}) < TC(x_{best})$  then
13:     $x_{best} \leftarrow x_{bestNeighbor}$ ;
14:  end if
15:   $x_{current} \leftarrow x_{bestNeighbor}$ ;
16:  Update the tabu list;
17: end while
18: Output: an improved solution:  $x_{best}$ .

```

TABLE I

THE NUMBER OF VEHICLES AND ORDERS OF DIFFERENT HW PROBLEMS

Problems	K	C	Q	Scale
HW1 - 8	5	50	15	small
HW9 - 6	5	100	15	small
HW17 - 24	20	300	15	small
HW25 - 32	20	500	15	small
HW33 - 40	50	1000	15	medium
HW41 - 48	50	2000	15	medium
HW49 - 56	100	3000	15	large
HW57 - 64	100	4000	15	large

real-life industrial scenarios, which have various practical constraints, such as docks, time windows, loading capacity, and LIFO loading. Thus, these DPDP benchmarks are very challenging and difficult to solve.

These HW problems (HW1-64) used in this study are derived from the Huawei supply chain systems and span a period of 30 days, providing a rich source of historical data. As depicted in Table I, the HW problems encompass four homogeneous vehicle quantity scales (5, 20, 50, and 100) and eight order quantity scales (50, 100, 300, 500, 1000, 2000, 3000, and 4000). In this paper, we classify these problems into three scales (small, medium, and large) according to the order quantity. Fig. 6 shows the distribution of newly generated orders throughout a day for three representative problems with different scales (HW1, HW33, HW64), in which the distributions of orders are calculated in 10-minute intervals. For the small-scale problem (HW1), the distribution of new orders is relatively sparse, with fewer orders per time interval within the 144 available time intervals in a day. In contrast, the medium-scale problem (HW33) exhibits a denser distribution of new orders throughout one day, accompanied by an increased number of orders. The large-scale problem (HW64) shows a denser distribution of new orders throughout one day and has a significantly larger number of new orders generated within each time interval.

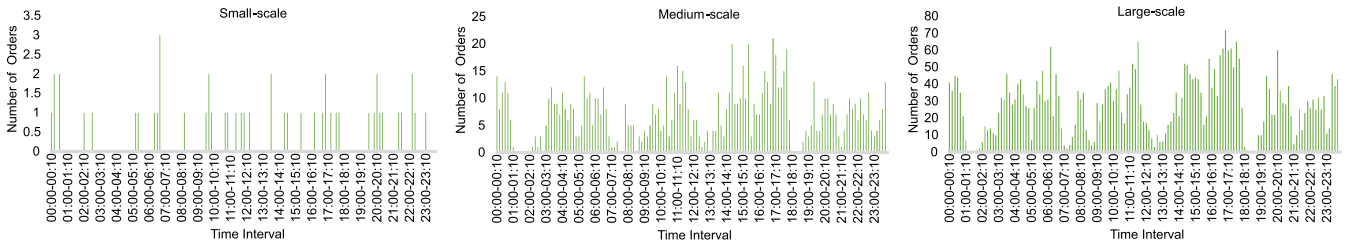


Fig. 6. The distributions of new orders generated by three representative HW problems with different scales.

Within each HW problem, multiple factories have the capability to dynamically generate delivery orders throughout the day. When addressing these HW problems, the objective is to assign all orders to a fleet of vehicles in a way that minimizes the total tardiness of orders and the average travel distance required to complete all orders. Therefore, the performance metric employed for the HW problems is the TC , as defined in Eq. (4), which is identical to the performance metric of the DPDP competition at ICAPS 2021.

B. Experimental Setup

To validate the effectiveness of our proposed MOEA/D-TS, it is compared with five up-to-date heuristic algorithms designed for solving HW problems, which include the HMTS [46], gold medal algorithm (Gold [31]), silver medal algorithm (Silver), bronze medal algorithm (Bronze), and baseline algorithm (Baseline). For comprehensive details regarding this competition, please refer to the official website.² Here, we provide a concise introduction to each compared algorithm:

HMTS: This algorithm is proposed for the heterogeneous fleet vehicle routing problem with three-dimensional loading constraints, utilizing a tabu search algorithm for vehicle routing optimization. This tabu search technique integrates several strategies such as a neighborhood structure, a fusion procedure, and an application of classical adaptive memory programming.

Gold: This algorithm adopts a variable neighborhood search (VNS) strategy, which consists of four local search strategies and one perturbation strategy. It continuously swaps pickup point clusters and delivery point clusters between different vehicle routes, which also adjusts order nodes within the same route to enhance the quality of the solution.

Silver: This algorithm converts the DPDP into a knapsack problem, ensuring that all order nodes are assigned to suitable knapsacks (routes) while accounting for all practical constraints.

Bronze: This algorithm initializes the route planning solution of DPDP using the CI algorithm and then further improves it using a destroy-reconstruct local search strategy.

Baseline: This method employs a traditional technique designed for static PDP [24] to address DPDP. Upon the release of each order, all incomplete pickup and delivery nodes are integrated into available vehicles to yield a feasible solution.

MOEA/D-TS and other compared algorithms process orders at 10-minute intervals. The allocation of orders from previous intervals directly influences the vehicle capacity and relative positioning of subsequent orders, thus establishing an interdependence between current and prior allocations. The algorithms operate within a 10-minute runtime, reflecting the practical constraints of industrial DPDP. For each instance in the HW problems, all the compared algorithms are executed independently 20 times.

The source codes for Gold, Silver, and Bronze are available on the following website.³ The Baseline algorithm's code can be found on another site.⁴ To ensure consistency and fairness, the parameters of the compared algorithms are set as suggested by their authors. The configuration parameters for MOEA/D-TS and the other five compared algorithms include 1) a penalty value of 1 hour for order tardiness, denoted as $\alpha = 10000/3600$ (derived from estimates within the Huawei logistics distribution system), 2) a limit of 6 docks per factory, 3) the vehicle's maximum load capacity Q restricted to 15, and 4) a constrained runtime of 10 minutes. These parameters are aligned with the standards set by the ICAPS 2021 DPDP competition. Moreover, for all test problems, MOEA/D-TS employs a population size (N) of 6 and a number of weight vectors in the neighborhood (T) of 2, with stopping criteria of I) a maximum of 50 iterations and II) a maximum runtime of 600 seconds.

Experiments in this study were executed on a system running the Ubuntu 16.04 LTS operating system, powered by an Intel(R) Core(TM) i7-6700 CPU at 3.40 GHz with 8 cores, and equipped with 9.7 GB of RAM.

C. Statistical Results and Discussion

In this section, we aim to enhance the clarity of the descriptions and analysis by categorizing the results into three groups based on the scales of the HW problems: small-scale, medium-scale, and large-scale. Table I details this classification, while Fig. 6 illustrates the order distribution for each category. Moreover, Tables II-IV provide the TC values obtained by all the algorithms across these scales. In these tables, the columns "B.Cost" and "Ave.Cost" represent the best TC value, the average TC value and its standard deviation from 20 independent runs of each algorithm under comparison. Notably, the best results are highlighted in bold. To ensure a statistically reliable conclusion, a Wilcoxon rank-sum test

²<https://competition.huaweicloud.com/information/1000041411/circumstance>

³<https://competition.huaweicloud.com/information/1000041411/Winning>

⁴https://github.com/huawei-noah/xingtian/tree/master/simulator/dpdp_competition

TABLE II
COMPARISON RESULTS OF GOLD, SILVER, BRONE, BASELINE, HMTS, AND MOEA/D-TS ON SMALL-SCALE HW PROBLEMS

Problem	C/K	Gold		Silver		Bronze		Baseline		HMTS		MOEA/D-TS	
		B.Cost	Ave.Cost	B.Cost	Ave.Cost	B.Cost	Ave.Cost	B.Cost	Ave.Cost	B.Cost	Ave.Cost	B.Cost	Ave.Cost
HW1	50/5	1.34E+2	1.34E+2 (0.00E+0) -	2.30E+3	2.30E+3 (0.00E+0) -	1.30E+2	1.30E+2 (0.00E+0) -	1.58E+5	1.58E+5 (0.00E+0) -	1.26E+2	1.30E+2 (4.03E+0) -	1.17E+2	1.17E+2 (5.48E-2)
HW2	50/5	9.56E+1	9.56E+1 (0.00E+0) -	3.05E+4	3.05E+4 (0.00E+0) -	9.14E+1	9.14E+1 (0.00E+0) -	8.98E+4	8.98E+4 (0.00E+0) -	9.24E+1	9.31E+1 (1.54E+0) -	8.87E+1	8.88E+1 (1.20E-1)
HW3	50/5	9.68E+1	9.68E+1 (0.00E+0) -	3.58E+4	3.58E+4 (0.00E+0) -	9.65E+1	9.65E+1 (0.00E+0) -	3.38E+4	3.38E+4 (0.00E+0) -	9.29E+1	9.29E+1 (0.00E+0) +	9.41E+1	9.41E+1 (0.00E+0)
HW4	50/5	9.46E+1	9.46E+1 (0.00E+0) +	5.49E+3	5.49E+3 (0.00E+0) -	1.04E+2	1.04E+2 (0.00E+0) -	4.18E+4	4.18E+4 (0.00E+0) -	9.51E+1	9.53E+1 (3.94E-1) +	1.01E+2	1.01E+2 (6.57E-2)
HW5	50/5	3.31E+3	3.31E+3 (2.11E+0) ~	1.69E+4	1.69E+4 (0.00E+0) -	5.45E+3	5.45E+3 (0.00E+0) -	1.47E+5	1.47E+5 (0.00E+0) -	3.31E+3	3.31E+3 (2.43E+0) ~	3.28E+3	3.30E+3 (1.44E+1)
HW6	50/5	1.05E+2	1.05E+2 (0.00E+0) ~	4.76E+3	4.76E+3 (0.00E+0) -	1.18E+2	1.18E+2 (0.00E+0) -	5.24E+4	5.24E+4 (0.00E+0) -	1.05E+2	1.07E+2 (1.08E+0) ~	1.05E+2	1.07E+2 (2.20E+0)
HW7	50/5	4.39E+3	4.39E+3 (0.00E+0) -	1.28E+4	1.28E+4 (0.00E+0) -	7.36E+3	7.36E+3 (0.00E+0) -	9.57E+4	9.57E+4 (0.00E+0) -	3.79E+3	4.27E+3 (2.69E+2) -	3.79E+3	4.13E+3 (1.94E+2)
HW8	50/5	6.88E+1	6.88E+1 (0.00E+0) -	7.97E+2	7.97E+2 (0.00E+0) -	7.69E+2	7.69E+2 (0.00E+0) -	3.88E+4	3.88E+4 (0.00E+0) -	6.57E+1	6.78E+1 (1.18E+0) -	6.31E+1	6.40E+1 (1.14E+0)
HW9	100/5	1.52E+2	1.52E+2 (0.00E+0) ~	1.81E+5	1.81E+5 (0.00E+0) -	8.48E+3	8.48E+3 (0.00E+0) -	2.12E+6	2.12E+6 (0.00E+0) -	1.59E+2	2.72E+3 (3.50E+3) -	1.51E+2	1.56E+2 (3.37E+0)
HW10	100/5	1.63E+5	1.78E+5 (1.47E+4) -	1.77E+6	1.77E+6 (0.00E+0) -	1.87E+5	1.87E+5 (0.00E+0) -	5.42E+6	5.42E+6 (0.00E+0) -	1.04E+5	1.12E+5 (1.23E+4) -	7.57E+4	8.88E+4 (9.05E+3)
HW11	100/5	1.98E+2	1.98E+2 (0.00E+0) -	1.81E+5	1.81E+5 (0.00E+0) -	3.04E+3	3.04E+3 (0.00E+0) -	1.92E+6	1.92E+6 (2.60E-10) -	1.72E+2	1.77E+2 (5.04E+0) -	1.61E+2	1.64E+2 (3.56E+0)
HW12	100/5	5.29E+4	5.29E+4 (0.00E+0) -	5.67E+5	5.67E+5 (0.00E+0) -	8.42E+4	8.42E+4 (0.00E+0) -	3.25E+6	3.25E+6 (0.00E+0) -	1.98E+4	3.39E+4 (1.17E+4) -	5.32E+3	5.32E+3 (0.00E+0)
HW13	100/5	7.18E+3	7.18E+3 (0.00E+0) -	2.20E+5	2.20E+5 (0.00E+0) -	2.77E+2	2.77E+2 (0.00E+0) -	2.50E+6	2.50E+6 (0.00E+0) -	1.89E+2	3.21E+3 (4.05E+3) -	1.69E+2	1.71E+2 (1.81E+0)
HW14	100/5	9.39E+3	9.39E+3 (0.00E+0) -	2.37E+5	2.37E+5 (0.00E+0) -	7.82E+3	7.82E+3 (0.00E+0) -	2.61E+6	2.61E+6 (0.00E+0) -	1.66E+2	1.59E+3 (2.11E+3) -	1.50E+2	1.58E+2 (4.42E+0)
HW15	100/5	1.35E+4	2.91E+4 (1.42E+4) ~	7.93E+5	7.93E+5 (0.00E+0) -	1.49E+5	1.49E+5 (0.00E+0) -	3.36E+6	3.36E+6 (0.00E+0) -	1.96E+4	4.67E+4 (1.81E+4) -	1.84E+4	2.16E+4 (4.41E+3)
HW16	100/5	1.68E+4	2.83E+4 (6.46E+3) ~	8.54E+5	8.54E+5 (0.00E+0) -	5.78E+4	5.78E+4 (0.00E+0) -	3.24E+6	3.24E+6 (0.00E+0) -	2.75E+4	3.12E+4 (2.69E+3) -	1.39E+4	1.75E+4 (2.08E+3)
HW17	300/20	8.17E+1	8.17E+1 (0.00E+0) -	9.65E+1	9.65E+1 (0.00E+0) -	4.11E+2	4.11E+2 (0.00E+0) -	1.31E+6	1.31E+6 (0.00E+0) -	8.10E+1	8.29E+1 (1.84E+0) -	7.44E+1	7.59E+1 (1.03E+0)
HW18	300/20	8.22E+1	8.22E+1 (0.00E+0) -	1.12E+2	1.12E+2 (0.00E+0) -	8.56E+3	8.56E+3 (0.00E+0) -	9.72E+5	9.72E+5 (0.00E+0) -	8.21E+1	8.37E+1 (1.92E+0) -	7.81E+1	7.92E+1 (7.71E-1)
HW19	300/20	1.09E+2	1.09E+2 (0.00E+0) -	4.78E+2	4.78E+2 (0.00E+0) -	4.15E+3	4.15E+3 (0.00E+0) -	1.93E+6	1.93E+6 (0.00E+0) -	1.08E+2	1.12E+2 (3.30E+0) -	1.01E+2	1.05E+2 (4.28E+0)
HW20	300/20	3.30E+3	3.30E+3 (0.00E+0) ~	3.98E+3	3.98E+3 (0.00E+0) -	1.62E+4	1.62E+4 (0.00E+0) -	2.10E+6	2.10E+6 (0.00E+0) -	3.53E+3	3.53E+3 (1.52E+0) ~	3.29E+3	3.44E+3 (1.40E+2)
HW21	300/20	1.13E+2	1.13E+2 (4.47E-3) -	2.34E+3	2.34E+3 (0.00E+0) -	1.85E+4	1.85E+4 (0.00E+0) -	2.60E+6	2.60E+6 (0.00E+0) -	1.00E+2	1.79E+3 (1.67E+3) -	9.31E+1	9.81E+1 (2.93E+0)
HW22	300/20	1.65E+3	1.65E+3 (0.00E+0) -	3.32E+3	3.32E+3 (0.00E+0) -	2.11E+4	2.11E+4 (0.00E+0) -	2.29E+6	2.29E+6 (0.00E+0) -	1.64E+3	3.16E+3 (8.49E+2) -	1.63E+3	1.64E+3 (1.63E+0)
HW23	300/20	1.05E+2	1.05E+2 (0.00E+0) ~	6.87E+3	6.87E+3 (0.00E+0) -	8.09E+2	8.09E+2 (0.00E+0) -	2.32E+6	2.32E+6 (0.00E+0) -	1.07E+2	1.08E+2 (1.54E+0) -	1.00E+2	1.04E+2 (2.84E+0)
HW24	300/20	9.44E+1	9.44E+1 (0.00E+0) -	1.29E+3	1.29E+3 (0.00E+0) -	2.00E+3	2.00E+3 (0.00E+0) -	1.44E+6	1.44E+6 (0.00E+0) -	8.69E+1	9.06E+1 (2.65E+0) -	8.43E+1	8.62E+1 (1.15E+0)
HW25	500/20	1.03E+4	1.03E+4 (0.00E+0) -	9.24E+4	9.24E+4 (0.00E+0) -	2.15E+4	2.15E+4 (0.00E+0) -	3.69E+7	3.69E+7 (0.00E+0) -	7.88E+3	9.43E+3 (9.31E+2) -	5.86E+3	6.06E+3 (2.14E+2)
HW26	500/20	8.95E+3	8.95E+3 (1.09E+0) -	2.75E+5	2.75E+5 (0.00E+0) -	5.40E+4	5.40E+4 (0.00E+0) -	3.54E+7	3.54E+7 (0.00E+0) -	8.57E+3	9.15E+3 (3.59E+2) -	3.84E+3	5.13E+3 (1.28E+3)
HW27	500/20	1.34E+2	1.34E+2 (0.00E+0) -	1.84E+4	1.84E+4 (0.00E+0) -	1.88E+4	1.88E+4 (0.00E+0) -	3.13E+7	3.13E+7 (0.00E+0) -	1.24E+2	1.30E+2 (3.78E+0) -	1.23E+2	1.24E+2 (5.62E-1)
HW28	500/20	7.10E+3	7.10E+3 (3.12E+0) +	9.44E+3	9.44E+3 (0.00E+0) -	1.49E+4	1.49E+4 (0.00E+0) -	2.86E+7	2.86E+7 (0.00E+0) -	7.10E+3	7.11E+3 (1.34E+1) ~	7.12E+3	7.12E+3 (3.38E+0)
HW29	500/20	6.40E+3	8.26E+3 (1.04E+3) -	1.63E+5	1.63E+5 (0.00E+0) -	4.24E+4	4.24E+4 (0.00E+0) -	3.64E+7	3.64E+7 (0.00E+0) -	6.17E+3	7.48E+3 (2.25E+3) -	1.67E+3	4.70E+3 (1.83E+3)
HW30	500/20	1.19E+2	1.19E+2 (4.47E-3) -	7.40E+4	7.40E+4 (0.00E+0) -	2.11E+4	2.11E+4 (0.00E+0) -	3.19E+7	3.19E+7 (0.00E+0) -	1.17E+2	1.20E+2 (2.28E+0) -	1.06E+2	1.08E+2 (2.26E+0)
HW31	500/20	2.26E+4	2.83E+4 (3.19E+3) -	1.47E+5	1.47E+5 (0.00E+0) -	2.33E+4	2.33E+4 (0.00E+0) -	3.80E+7	3.80E+7 (0.00E+0) -	1.57E+4	1.83E+4 (4.80E+3) ~	1.27E+4	1.43E+4 (1.76E+3)
HW32	500/20	7.67E+3	7.67E+3 (0.00E+0) -	6.04E+4	6.04E+4 (0.00E+0) -	1.96E+4	1.96E+4 (0.00E+0) -	3.63E+7	3.63E+7 (0.00E+0) -	7.66E+3	7.66E+3 (1.99E+0) -	7.36E+3	7.56E+3 (1.22E+2)
best/all	4/32	5/32	0/32	0/32	0/32	0/32	0/32	0/32	0/32	4/32	1/32	28/32	26/32
+/-/~		2/23/7		0/32/0		0/32/0		0/32/0		0/32/0	2/25/5		—

TABLE III
COMPARISON RESULTS OF GOLD, SILVER, BRONE, BASELINE, HMTS, AND MOEA/D-TS ON MEDIUM-SCALE HW PROBLEMS

Problem	C/K	Gold		Silver		Bronze		Baseline		HMTS		MOEA/D-TS	
		B.Cost	Ave.Cost	B.Cost	Ave.Cost	B.Cost	Ave.Cost	B.Cost	Ave.Cost	B.Cost	Ave.Cost	B.Cost	Ave.Cost
HW33	1000/50	1.59E+3	1.59E+3 (4.47E-3) -	7.03E+3	7.03E+3 (0.00E+0) -	9.50E+4	9.50E+4 (0.00E+0) -	3.86E+7	3.86E+7 (0.00E+0) -	7.42E+1	1.59E+3 (9.79E+2) -	6.91E+1	6.76E+2 (8.30E+2)
HW34	1000/50	1.05E+4	1.05E+4 (3.45E-1) -	1.05E+4	1.05E+4 (0.00E+0) -	5.44E+4	5.44E+4 (0.00E+0) -	3.98E+7	3.98E+7 (0.00E+0) -	1.03E+4	1.18E+4 (1.37E+3) -	5.51E+3	6.81E+3 (9.66E+2)
HW35	1000/50	3.44E+3	3.44E+3 (0.00E+0) -	1.58E+4	1.58E+4 (0.00E+0) -	1.04E+5	1.04E+5 (0.00E+0) -	5.11E+7	5.11E+7 (0.00E+0) -	3.44E+3	3.44E+3 (2.17E+0) -	7.89E+1	2.48E+3 (1.47E+3)
HW36	1000/50	1.75E+4	1.75E+4 (0.00E+0) -	2.32E+4	2.32E+4 (0.00E+0) -	1.13E+5	1.13E+5 (0.00E+0) -	4.67E+7	4.67E+7 (0.00E+0) -	1.22E+4	1.65E+4 (2.45E+3) ~	1.45E+4	1.64E+4 (1.09E+3)
HW37	1000/50	1.12E+4	1.12E+4 (0.00E+0) ~	5.14E+3	5.14E+3 (0.00E+0) +	9.57E+4	9.57E+4 (0.00E+0) -	4.67E+7	4.67E+7 (0.00E+0) -	7.77E+3	1.05E+4 (2.19E+3) ~	7.30E+3	9.27E+3 (2.05E+3)
HW38	1000/50	1.50E+4	1.56E+4 (3.89E+2) -	3.01E+4	3.01E+4 (0.00E+0) -	9.42E+4	9.42E+4 (0.00E+0) -	4.70E+7	4.70E+7 (0.00E+0) -	1.37E+4	1.69E+4 (2.74E+3) -	6.79E+3	1.25E+4 (3.18E+3)
HW39	1000/50	1.49E+4	1.49E+4 (0.00E+0) +	2.23E+4	2.23E+4 (0.00E+0) -	8.52E+4	8.52E+4 (0.00E+0) -	4.43E+7	4.43E+7 (0.00E+0) -	1.52E+4	1.75E+4 (1.75E+3) ~	1.70E+4	1.74E+4 (2.51E+2)
HW40	1000/50	1.02E+4	1.02E+4 (0.00E+0) ~	2.43E+4	2.43E+4 (0.00E+0) -	1.17E+5	1.17E+5 (0.00E+0) -	3.73E+7	3.73E+7 (0.00E+0) -	1.26E+4	1.32E+4 (3.44E+2) -	9.33E+3	9.85E+3 (4.43E+2)
HW41	2000/50	2.93E+4	2.93E+4 (0.00E+0) -	1.24E+5	1.24E+5 (0.00E+0) -	5.49E+5	5.49E+5 (0.00E+0) -	4.48E+8	4.48E+8 (0.00E+0) -	2.56E+4	3.74E+4 (7.77E+3) -	2.04E+4	2.14E+4 (7.06E+2)
HW42	2000/50	3.59E+4	4.45E+4 (7.94E+3) -	1.64E+5	1.64E+5 (0.00E+0) -	6.01E+5	6.01E+5 (0.00E+0) -	4.23E+8	4.23E+8 (0.00E+0) -	3.09E+4	4.64E+4 (1.52E+4) -	1.92E+4	2.05E+4 (9.21E+2)
HW43	2000/50	6.05E+4	7.55E+4 (2.32E+4) -	1.67E+5	1.67E+5 (0.00E+0) -	5.51E+5	5.51E+5 (0.00E+0) -	4.24E+8	4.24E+8 (0.00E+0) -	4.52E+4	7.30E+4 (3.03E+4) -	2.46E+4	2.99E+4 (5.77E+3)
HW44	2000/50	4.96E+4	7.09E+4 (1.23E+4) -	1.91E+5	1.91E+5 (0.00E+0) -	7.15E+5	7.15E+5 (0.00E+0) -	4.27E+8	4.27E+8 (0.00E+0) -	5.40E+4	8.57E+4 (3.43E+4) -	4.33E+4	4.48E+4 (1.43E+3)
HW45	2000/50	3.63E+4	4.32E+4 (9.61E+3) -	1.81E+5	1.81E+5 (0.00E+0) -	6.16E+5	6.16E+5 (0.00E+0) -	4.26E+8	4.26E+8 (0.00E+0) -	3.14E+4	5.42E+4 (2.67E+4) -	1.68E+4	1.82E+4 (1.43E+3)
HW46	2000/50	3.09E+4	3.09E+4 (1.30E-1) -	1.76E+5	1.76E+5 (0.00E+0) -	7.98E+5	7.98E+5 (0.00E+0) -	4.50E+8	4.50E+8 (0.00E+0) -	1.57E+4	2.89E+4 (1.29E+4) -	1.33E+4	1.58E+4 (2.50E+3)
HW47	2000/50	2.68E+4	2.68E+4 (0.00E+0) -	1.04E+5	1.04E+5 (0.00E+0) -	6.03E+5	6.03E+5 (0.00E+0) -	4.24E+8	4.24E+8 (0.00E+0) -	1.82E+4	2.11E+4 (1.67E+3) ~	1.26E+4	1.76E+4 (4.00E+3)
HW48	2000/50	1.33E+4	1.45E+4 (1.06E+3) ~	1.20E+5	1.20E+5 (0.00E+0) -	7.82E+5	7.82E+5 (0.00E+0) -	4.32E+8	4.32E+8 (0.00E+0) -	1.30E+4	1.55E+4 (1.52E+3) -	1.13E+4	1.32E+4 (1.82E+3)
best/all	1/16	1/16	1/16	1/16	0/16	0/16	0/16	0/16	0/16	1/16	0/16	13/16	14/16
+/-/~		1/12/3		1/15/0		0/16/0		0/16/0		0/12/4		—	—

with a significance level of 0.05 is conducted on the TC values to indicate statistically significant differences. Specifically, the symbols “+”, “-”, and “~” behind each standard deviation indicate that the corresponding compared algorithm performs better than, worse than, and similarly to MOEA/D-TS. In the penultimate row of each table, “best/all” provides a summary of the best results of each algorithm based on “B.Cost” and “Ave.Cost”, respectively. Moreover, “+/-/~” in the last row of each table summarizes the numbers

TABLE IV
COMPARISON RESULTS OF GOLD, SILVER, BRONZE, BASELINE, HMTS, AND MOEA/D-TS ON LARGE-SCALE HW PROBLEMS

Problem	C/K	Gold		Silver		Bronze		Baseline		HMTS		MOEA/D-TS	
		B.Cost	Ave.Cost	B.Cost	Ave.Cost	B.Cost	Ave.Cost	B.Cost	Ave.Cost	B.Cost	Ave.Cost	B.Cost	Ave.Cost
HW49	3000/100	1.23E+6	1.43E+6 (1.39E+5) -	1.18E+6	1.18E+6 (0.00E+0) -	1.59E+6	1.59E+6 (0.00E+0) -	9.20E+8	9.20E+8 (0.00E+0) -	1.50E+6	1.57E+6 (5.22E+4) -	5.75E+5	7.41E+5 (1.55E+5)
HW50	3000/100	7.44E+5	7.95E+5 (7.84E+4) -	5.72E+5	5.72E+5 (0.00E+0) ~	2.51E+6	2.51E+6 (0.00E+0) -	9.20E+8	9.20E+8 (0.00E+0) -	1.18E+6	1.37E+6 (1.52E+5) -	5.62E+5	6.57E+5 (5.98E+4)
HW51	3000/100	7.44E+4	2.07E+5 (1.38E+5) ~	5.15E+5	5.15E+5 (0.00E+0) -	1.34E+6	1.34E+6 (0.00E+0) -	9.13E+8	9.13E+8 (0.00E+0) -	2.98E+5	3.64E+5 (5.12E+4) -	5.27E+4	1.11E+5 (6.77E+4)
HW52	3000/100	7.01E+5	7.53E+5 (4.25E+4) -	4.48E+5	4.48E+5 (0.00E+0) -	2.03E+6	2.03E+6 (0.00E+0) -	9.20E+8	9.20E+8 (0.00E+0) -	6.90E+5	8.79E+5 (1.06E+5) -	2.70E+5	3.76E+5 (6.48E+4)
HW53	3000/100	8.47E+4	9.56E+4 (1.47E+4) ~	3.17E+5	3.17E+5 (0.00E+0) -	1.93E+6	1.93E+6 (0.00E+0) -	9.55E+8	9.55E+8 (0.00E+0) -	3.02E+5	5.29E+5 (2.10E+5) -	1.48E+4	6.61E+4 (5.51E+4)
HW54	3000/100	1.93E+6	1.98E+6 (6.16E+4) ~	1.63E+6	1.63E+6 (0.00E+0) -	2.00E+6	2.00E+6 (0.00E+0) ~	9.11E+8	9.11E+8 (0.00E+0) -	2.05E+6	2.42E+6 (3.09E+5) -	1.40E+6	1.77E+6 (2.71E+5)
HW55	3000/100	2.61E+5	3.02E+5 (2.79E+4) ~	7.30E+5	7.30E+5 (0.00E+0) -	1.98E+6	1.98E+6 (0.00E+0) -	8.97E+8	8.97E+8 (0.00E+0) -	5.71E+5	8.46E+5 (1.71E+5) -	7.11E+4	2.07E+5 (1.51E+5)
HW56	3000/100	1.92E+6	1.99E+6 (6.09E+4) -	1.84E+6	1.84E+6 (0.00E+0) ~	2.16E+6	2.16E+6 (0.00E+0) -	9.45E+8	9.45E+8 (0.00E+0) -	1.90E+6	1.95E+6 (3.33E+4) -	1.44E+6	1.71E+6 (2.07E+5)
HW57	4000/100	9.83E+6	1.01E+7 (2.75E+5) -	2.24E+7	2.24E+7 (0.00E+0) -	1.65E+7	1.65E+7 (0.00E+0) -	1.83E+9	1.83E+9 (0.00E+0) -	9.22E+6	1.05E+7 (9.40E+5) ~	9.05E+6	9.17E+6 (1.12E+5)
HW58	4000/100	9.37E+6	1.01E+7 (8.15E+5) -	1.32E+7	1.32E+7 (0.00E+0) -	1.12E+7	1.12E+7 (0.00E+0) -	1.81E+9	1.81E+9 (0.00E+0) -	8.57E+6	9.60E+6 (6.51E+5) -	7.62E+6	8.34E+6 (6.41E+5)
HW59	4000/100	7.77E+6	8.81E+6 (6.02E+5) ~	1.24E+7	1.24E+7 (0.00E+0) -	9.54E+6	9.54E+6 (0.00E+0) -	1.90E+9	1.90E+9 (0.00E+0) -	9.04E+6	1.03E+7 (7.56E+5) -	8.00E+6	8.72E+6 (5.02E+5)
HW60	4000/100	1.07E+7	1.09E+7 (1.52E+5) -	2.15E+7	2.15E+7 (0.00E+0) -	1.32E+7	1.32E+7 (0.00E+0) -	1.83E+9	1.83E+9 (0.00E+0) -	1.08E+7	1.21E+7 (1.21E+6) -	9.33E+6	1.02E+7 (5.73E+5)
HW61	4000/100	6.03E+6	7.18E+6 (7.96E+5) -	9.10E+6	9.10E+6 (0.00E+0) -	7.21E+6	7.21E+6 (0.00E+0) -	1.81E+9	1.81E+9 (0.00E+0) -	7.02E+6	7.88E+6 (8.06E+5) -	4.39E+6	5.13E+6 (1.12E+6)
HW62	4000/100	8.80E+6	9.12E+6 (2.34E+5) +	1.45E+7	1.45E+7 (0.00E+0) -	1.09E+7	1.09E+7 (0.00E+0) ~	1.87E+9	1.87E+9 (0.00E+0) -	1.22E+7	1.25E+7 (3.15E+5) -	9.99E+6	1.09E+7 (6.79E+5)
HW63	4000/100	1.33E+7	1.76E+7 (2.69E+6) ~	3.09E+7	3.09E+7 (0.00E+0) -	2.50E+7	2.50E+7 (0.00E+0) -	1.91E+9	1.91E+9 (0.00E+0) -	1.21E+7	1.45E+7 (1.73E+6) ~	1.46E+7	1.58E+7 (1.14E+6)
HW64	4000/100	1.57E+7	1.71E+7 (8.68E+5) -	2.54E+7	2.54E+7 (0.00E+0) -	2.74E+7	2.74E+7 (0.00E+0) -	1.96E+9	1.96E+9 (0.00E+0) -	1.48E+7	1.56E+7 (9.54E+5) -	1.21E+7	1.35E+7 (1.32E+6)
best/all		2/16	1/16	0/16	1/16	0/16	0/16	0/16	0/16	1/16	1/16	13/16	13/16
+/-/~			1/9/6		0/1/3		0/1/4/2		0/16/0		0/14/2		—

and 32 problems, respectively, while it underperforms in only 2 problems against both HMTS and Gold.

In these small-scale HW problems, Baseline distinctly underperforms the other compared algorithms, especially when assessing both the best TC value and the average TC value. Such underperformance arises from the underlying interdependence in order allocations of different intervals within the industrial DPDP. Moreover, the multifaceted nature of DPDP, which encompasses dynamic attributes, expansive variables, and myriad intricate constraints, exacerbates this challenge. As a result, Baseline consumes an excessive amount of computational costs, as it solves the DPDP by iteratively reoptimizing each static PDP. This leads to decreased efficiency and suboptimal results. These findings also confirm the efficacy of heuristic algorithms, which can seamlessly integrate new orders into existing routes to reduce computational costs.

2) *Comparisons on Medium-Scale HW Problems:* Table III presents the experimental results of “B.Cost” and “Ave.Cost” obtained from 20 independent runs of all the compared algorithms on 16 medium-scale HW problems. As observed from Table III, MOEA/D-TS performs superiorly on most of the medium-scale HW problems, which validates its advantages over the other five compared algorithms. Specifically, regarding “B.Cost”, MOEA/D-TS achieves the best results in 13 out of 16 problems, whereas HMTS, Gold, and Silver only perform best in 1 problem each, and Bronze and Baseline do not achieve the best performance in any problem, which validates the superiority of MOEA/D-TS for solving medium-scale HW problems. Particularly, MOEA/D-TS performs markedly better than the compared algorithms on HW34, HW35, HW38, and HW40.

Concerning “Ave.Cost”, MOEA/D-TS obtains the best results in 14 out of 16 problems. The Wilcoxon rank-sum test reveals that, compared to HMTS, Gold, Silver, Bronze, and Baseline, MOEA/D-TS outperforms in 12, 12, 15, 16, and 16 out of 16 problems, respectively, and exhibits underperformance in 0, 1, 1, 0, and 0 cases, respectively. These results confirm the robustness of MOEA/D-TS. In all the medium-scale HW problems, the mean TC values for MOEA/D-TS, HMTS, Gold, Silver, Bronze, and Baseline stand at 1.61E+4, 2.84E+4, 2.63E+4, 8.54E+4, 3.73E+5,

and 2.38E+8, respectively. Hence, MOEA/D-TS achieves the best mean TC value for these medium-scale HW problems. In comparison to HMTS, Gold, Silver, Bronze, and Baseline, MOEA/D-TS improves the TC metrics by 43.40%, 38.93%, 81.20%, 95.70%, and 99.99%, respectively.

In these medium-scale problems, MOEA/D-TS distinctly outperforms HMTS, Gold, Silver, Bronze, and Baseline when considering the “B.Cost” and “Ave.Cost” metrics. This performance gap stems from the fact that these five compared algorithms only optimize a single weighted objective considering the total tardiness and average travel distance. Although these algorithms can generate feasible solutions for intricate DPDPs, they are susceptible to being trapped in local optima. In contrast, MOEA/D-TS formulates the original DPDP as an MOP with two distinct objectives (f_1 and f_2). This MOP is further decomposed into multiple subproblems, which are addressed cooperatively by using both crossover and tabu search. This strategy not only improves the diversity of the solutions but also avoids becoming trapped in local optima.

3) *Comparisons on Large-Scale HW Problems:* According to Table I, each large-scale HW problem comprises a large number of orders, ranging from 3000 to 4000. Moreover, the dense distribution of the orders associated with each large-scale HW problem is depicted in Fig. 6, indicating their dynamic nature. Therefore, tackling large-scale HW problems is notably challenging.

Table IV displays the experimental results from 20 independent runs of all the compared algorithms on 16 large-scale HW problems (HW49-64). These results validate the advantages of MOEA/D-TS over the other five compared algorithms in most large-scale HW problems. In “B.Cost”, MOEA/D-TS achieves the best results in 13 out of 16 problems, while only Gold performs best in 2 problems (HW59 and HW62), and HMTS in 1 problem (HW63). Based on “Ave.Cost”, MOEA/D-TS obtains the best results in 13 out of 16 problems, while HMTS, Gold, Silver, Bronze, and Baseline achieve the best performances in 1, 1, 1, 0, and 0 problems, respectively. In detailed comparisons, MOEA/D-TS outperforms HMTS, Gold, Silver, Bronze, and Baseline in 14, 9, 13, 14, and 16 problems, respectively. In contrast, among the five competitors, only Gold surpasses MOEA/D-TS in HW62. In the large-scale HW

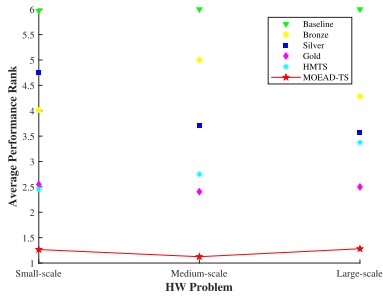


Fig. 7. Illustration of the average performance ranks over all scales of HW problems for the compared algorithms.

problems, the average TC values for MOEA/D-TS, HMTS, Gold, Silver, Bronze, and Baseline are $5.46\text{E}+6$, $6.43\text{E}+6$, $6.16\text{E}+6$, $9.79\text{E}+6$, $8.53\text{E}+6$, and $1.39\text{E}+9$, respectively. Compared to HMTS, Gold, Silver, Bronze, and Baseline, MOEA/D-TS improves the TC metric by 15.06%, 11.30%, 44.24%, 36.01%, and 99.61%, respectively.

MOEA/D-TS exhibits a promising capability in addressing HW problems. Its success is rooted in its approach: formulating the original DPDP as an MOP and subsequently decomposing the MOP into multiple subproblems. It promotes solution's diversity via crossover and exploits a promising solution domain by using a tabu search strategy. Thus, compared to its competitors, MOEA/D-TS shows a marked proficiency in identifying optimal regions within the solution space, enhancing the probability of finding superior solutions.

4) *Further Discussion and Analysis of the Performance:* To visually show the comparison of the evaluated algorithms, their average performance ranks on each scale of HW problems are determined using Friedman's test [63] based on the TC values, as depicted in Fig. 7. Please note that a lower rank of an algorithm implies its superior performance. As shown in Fig. 7, MOEA/D-TS performs best on all scales of HW problems.

MOEA/D-TS exhibits marginally superior performance on medium-scale HW problems compared to both small-scale and large-scale HW problems, as illustrated in Fig. 7. Specifically, the small-scale HW problems, as shown in Fig. 6, possess a relatively small number of orders generated dynamically in each interval, which are more straightforward to solve, and MOEA/D-TS does not easily fall into local optima. Nevertheless, MOEA/D-TS improves the diversity of solutions by crossover, which consumes additional resources and fails to thoroughly converge to the optimal solution region under limited computational resources. For large-scale HW problems, the vast quantity of orders generated within each interval results in highly complex solution spaces, causing MOEA/D-TS to become trapped in suboptimal solution regions. Conversely, medium-scale HW problems yield a moderate number of dynamically generated orders in each interval, in which MOEA/D-TS can well balance the diversity and convergence of the population by the crossover process and tabu search. Therefore, MOEA/D-TS obtained superior performance on medium-scale HW problems compared to both small-scale and large-scale HW problems. Nevertheless, MOEA/D-TS is still significantly superior to the other

compared algorithms in solving small-scale and large-scale HW problems.

In addition, MOEA/D-TS significantly outperforms HMTS in three HW scale problems, illustrating that reliance solely on the tabu search algorithm is insufficient for effectively addressing the industrial DPDP. This further demonstrates the efficacy of MOEA/D-TS, which leverages multiobjectivization to reformulate the original DPDP into an MOP, subsequently enhancing solution's diversity and quality through a combination of a crossover operator and tabu search, after decomposing the MOP into multiple subproblems. Moreover, the average performance rank of Bronze is lower than that of Silver in small-scale HW problems, but it deteriorates further in medium-scale and large-scale problems. Several factors contribute to this trend. When applying the destroy-reconstruct strategy, Bronze inserts each pair of pickup and delivery nodes into a route adjacently, which leads to a constrained search space and increases the likelihood of falling into local optima. Bronze outperforms Silver in small-scale HW problems due to their relatively smaller solution spaces. However, as the scale of HW problems increases, Bronze becomes more susceptible to local optima, while Silver demonstrates an advantage in addressing these problems with larger scales.

D. More Discussions

1) *Effectiveness of Each Component in MOEA/D-TS:* To assess the effectiveness of our proposed methods, we conducted a comparative analysis of MOEA/D-TS against its five variants:

- NO-ST: MOEA/D-TS without using the mechanism of solution transfer from previous intervals.
- NO-DF: MOEA/D-TS without using our decomposition-based framework.
- NO-CT: MOEA/D-TS without using our crossover operator and tabu search.
- NO-CR: MOEA/D-TS without using our crossover operator.
- NO-TS: MOEA/D-TS without using our tabu search.

Please note that the specified component is removed from MOEA/D-TS in the corresponding variant. To ensure a fair comparison, each algorithm was executed independently 20 times across all scales of HW problems, with the parameter settings of all variants aligned with those of MOEA/D-TS. Due to space limitations, detailed results for "B.Cost" and "Ave.Cost" across small-scale, medium-scale, and large-scale HW problems are presented in Tables A.I to A.III of the Supplementary Material. Table V summarizes the comparison results of MOEA/D-TS and its compared variants across the HW problems with different scales, while Fig. 8 offers a comprehensive visual comparison of their average performance ranks, based on the TC values. From the statistical results (+/-/~), MOEA/D-TS significantly outperforms the five variants on each scale of HW problems, which validates the effectiveness of each proposed component.

Although NO-ST underperforms MOEA/D-TS in small to medium-scale HW problems, it still secures the third rank among the six algorithms, with ranks of 3.19 and 3.06,

TABLE V
SUMMARY OF THE COMPARISONS OF MOEA/D-TS AND ITS VARIANTS ON ALL SCALES OF HW PROBLEMS BY TC

problem	NO-ST		NO-DF		NO-CT		NO-TS		NO-CR		MOEA/D-TS	
	+/-/~	Rank	+/-/~	Rank	+/-/~	Rank	+/-/~	Rank	+/-/~	Rank	+/-/~	Rank
small-scale	1/22/9	3.19	2/26/4	4.89	0/26/6	5.13	2/25/5	3.61	0/22/10	2.77	/	1.42
medium-scale	0/13/3	3.06	0/14/2	4.81	0/13/3	5.31	0/11/5	3.69	0/12/4	2.69	/	1.44
large-scale	0/16/0	6.00	0/16/0	4.13	0/16/0	4.88	0/16/0	3.00	0/10/6	1.88	/	1.13

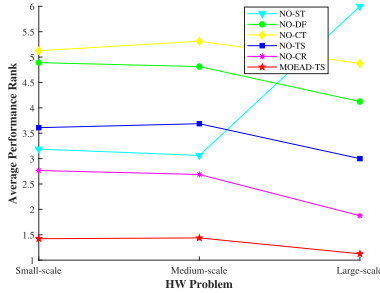


Fig. 8. Illustration of the average performance ranks over all scales of HW problems for the compared variants.

respectively. However, the efficacy of NO-ST significantly deteriorates in large-scale HW problems, ranking last among its counterparts. This deterioration is attributed to the vast number of orders in each interval of large-scale HW problems, where initializing solutions from scratch for each interval consumes significant computational resources, thereby severely limiting the capacity for subsequent optimization of the population. Thus, these observations highlight the efficiency of the solution transfer mechanism from previous intervals.

Absent the multiobjective decomposition-based framework, NO-DF exhibits relatively poor performance across all scales of HW problems, with ranks of 4.89, 4.81, and 4.13, respectively. This underperformance stems from a lack of cooperation among individuals in the population, resulting in a scarcity of global information guidance and a propensity for premature convergence to local optima. The distinct superiority of MOEA/D-TS over NO-DF underscores the significant advantage of our decomposition-based framework in solving industrial DPDPs.

In NO-CR, the absence of our crossover operator to foster diversity leads to a tendency toward local optima, resulting in underperformance compared to MOEA/D-TS, with ranks of 2.77, 2.69, and 1.88 across all scales of HW problems. This underlines the pivotal role of the crossover operator in augmenting the exploration capabilities of MOEA/D-TS. Meanwhile, without the tabu search for population optimization, NO-TS exhibits considerably inferior performance, with ranks of 3.61, 3.69, and 3.00, which is attributed to its insufficient exploitation within the solution space. This accentuates the indispensable contribution of tabu search in enhancing the exploitation efficiency of MOEA/D-TS. Moreover, NO-CT's absence of both the crossover operator and tabu search results in the overall worst performance across all compared algorithms, with ranks of 5.13, 5.31, and 4.88 for small, medium, and large-scale HW problems, respectively. This emphasizes the significant contribution of integrating both the crossover operator's exploration capability and tabu search's exploitation efficiency in enhancing the performance of MOEA/D-TS.

2) *Sensitivity Analysis*: This section investigates the sensitivity of the parameter T employed in MOEA/D-TS, which controls the diversity of offspring generated from the crossover process. In this experiment, the parameter is tested with values of $T = 1, 2, 3, 4$, and 5 , while maintaining other parameters in MOEA/D-TS consistent with previous experiments. The medium-scale HW problems mentioned earlier are utilized, and Friedman's test [63] is employed to conduct a statistical analysis on the TC values. Table A.IV of the Supplementary Material presents the values of "B.Cost" and "Ave.Cost" achieved by MOEA/D-TS using varying T values when applied to medium-scale HW problems. Fig. S1 of the Supplementary Material illustrates their average performance ranks on the TC values across each test problem. It is evident that the average performance ranks of MOEA/D-TS gradually improve from $T = 1$ to $T = 2$, followed by a decrease from $T = 2$ to $T = 5$. Specifically, when T is set too small (i.e., $T = 1$), only a few solutions are selected to generate offspring during the crossover process, potentially leading to insufficient diversity in the evolutionary process; however, when T is set relatively large (i.e., $T = 3, 4$ and 5), the offspring generated through crossover exhibit greater diversity, which can hinder their convergence to the optimal solution region under limited computational resources. Thus, T is recommended to be set to 2, as MOEA/D-TS consistently achieves the statistically best results in terms of "B.Cost" and "Ave.Cost" across most test cases with this setting.

3) *Generalization Validation*: To evaluate its generalizability and effectiveness, we employ MOEA/D-TS to address the vehicle routing problem with simultaneous pickup-delivery and time windows (VRPPDT) [64]. A large-scale VRPPDT dataset [13] is also adopted as a benchmark, which is obtained from the logistics distribution system of Jingdong (JD). In this system, logistics companies not only need to deliver goods to customers, but also have to collect defective goods or goods for repair, reuse or proper disposal, all within predetermined time windows. The specific settings of the large-scale JD dataset are shown in Table A.V of the Supplementary Material due to page limitations. The objective of VRPPDT is to minimize a weighted cost (WC) [13], which is defined as the sum of dispatching costs and transportation costs for the vehicles.

To verify the effectiveness of the proposed algorithm in solving large-scale VRPPDTs, this paper compares MOEA/D-TS with five recently proposed algorithms (MATE [13], EMA [1], CCMO [47], VNSME [31], and MBEA [48]) using the large-scale JD dataset. Each algorithm is independently executed 20 times on 20 problems within the large-scale JD dataset, totaling 18,000 evaluations per algorithm. Here, WC is used as the evaluation metric to evaluate the performance of

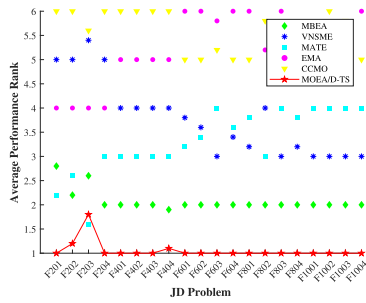


Fig. 9. Illustration of the average performance ranks across the Jingdong dataset for the compared algorithms.

each algorithm in solving VRPPDT. Due to space constraints, the comparison results of the WC values for all the compared algorithms are displayed in Table A.VI of the Supplementary Material. Moreover, in Fig. 9, the average performance rankings for all JD problems are presented by utilizing Friedman's test [63] based on the WC values, where the ranks of MOEA/D-TS are visually linked by a red line for improved clarity. MOEA/D-TS outperforms the other algorithms on the majority of JD problems, as observed from Fig. 9. MOEA/D-TS also obtains the best results in 19 out of 20 problems, as shown in Table A.VI of the Supplementary Material. This is because our algorithm consistently increases population's diversity by transforming the original DPDP into an MOP and then decomposing the MOP into multiple subproblems. Moreover, MOEA/D-TS effectively balances the diversity and convergence of the population by integrating the crossover process with local search when tackling each subproblem, thereby facilitating the discovery of higher-quality solutions. Therefore, this experiment offers empirical proof of the generalization capability and effectiveness of MOEA/D-TS in various practical scenarios, especially in solving VRPPDT.

V. CONCLUSION

In this study, we introduce a decomposition-based multi-objective evolutionary algorithm tailored to address industrial DPDP variants, considering constraints such as docks, time windows, loading capacity, and last-in-first-out loading. First, the DPDP is formulated as an MOP with dual subobjectives, which is then decomposed into multiple subproblems. These subproblems are addressed simultaneously and collaboratively by applying a crossover operation to enhance the diversity of solutions, followed by an efficient tabu search to accelerate the convergence. Ultimately, an optimal solution with the minimum TC value from the MOP population is selected for the original DPDP. The efficiency and effectiveness of MOEA/D-TS are demonstrated through simulations on 64 industrial logistics test problems provided by Huawei in ICAPS 2021. Furthermore, we validated the efficacy of each component of MOEA/D-TS via ablation experiments. The experiments have also validated that MOEA/D-TS outperforms five recently proposed algorithms when solving most cases of 20 real-world large-scale distribution problems within JD Logistics, which verifies the generalization capability and effectiveness of MOEA/D-TS.

To improve the performance of MOEA/D-TS for industrial DPDP variants, we plan to integrate both a delayed delivery

function and a predictive order strategy in future research. This strategic direction stems from the observed daily distribution patterns of orders within these industrial DPDP variants. Moreover, we intend to investigate the potential application of deep reinforcement learning techniques as an innovative approach to tackle DPDP variants in future studies.

REFERENCES

- [1] L. Feng et al., "Solving generalized vehicle routing problem with occasional drivers via evolutionary multitasking," *IEEE Trans. Cybern.*, vol. 51, no. 6, pp. 3171–3184, Jun. 2021.
- [2] R. Kaewpuang, S. Sawadsitang, D. Niyato, and H. Yu, "Evolutionary carrier selection for shared truck delivery services," *IEEE Trans. Veh. Technol.*, vol. 72, no. 5, pp. 6778–6782, 2023.
- [3] L. Feng et al., "Explicit evolutionary multitasking for combinatorial optimization: A case study on capacitated vehicle routing problem," *IEEE Trans. Cybern.*, vol. 51, no. 6, pp. 3143–3156, Jun. 2021.
- [4] İ. İlhan, "An improved simulated annealing algorithm with crossover operator for capacitated vehicle routing problem," *Swarm Evol. Comput.*, vol. 64, Jul. 2021, Art. no. 100911.
- [5] Z. Cao, H. Guo, J. Zhang, D. Niyato, and U. Fastenrath, "Improving the efficiency of stochastic vehicle routing: A partial Lagrange multiplier method," *IEEE Trans. Veh. Technol.*, vol. 65, no. 6, pp. 3993–4005, Jun. 2016.
- [6] Y.-H. Jia, Y. Mei, and M. Zhang, "A bilevel ant colony optimization algorithm for capacitated electric vehicle routing problem," *IEEE Trans. Cybern.*, vol. 52, no. 10, pp. 10855–10868, Oct. 2022.
- [7] Y.-H. Jia, Y. Mei, and M. Zhang, "Confidence-based ant colony optimization for capacitated electric vehicle routing problem with comparison of different encoding schemes," *IEEE Trans. Evol. Comput.*, vol. 26, no. 6, pp. 1394–1408, Dec. 2022.
- [8] X. Xiang, Y. Tian, X. Zhang, J. Xiao, and Y. Jin, "A pairwise proximity learning-based ant colony algorithm for dynamic vehicle routing problems," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 6, pp. 5275–5286, Jun. 2022.
- [9] J.-Y. Li et al., "A multipopulation multiobjective ant colony system considering travel and prevention costs for vehicle routing in COVID-19-like epidemics," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 12, pp. 25062–25076, Dec. 2022.
- [10] M. W. P. Savelsbergh and M. Sol, "The general pickup and delivery problem," *Transp. Sci.*, vol. 29, no. 1, pp. 17–29, 1995.
- [11] G. Berbeglia, J.-F. Cordeau, and G. Laporte, "Dynamic pickup and delivery problems," *Eur. J. Oper. Res.*, vol. 202, no. 1, pp. 8–15, Apr. 2010.
- [12] G. Berbeglia, J.-F. Cordeau, I. Gribkovskaia, and G. Laporte, "Static pickup and delivery problems: A classification scheme and survey," *TOP*, vol. 15, no. 1, pp. 1–31, Jul. 2007.
- [13] S. Liu, K. Tang, and X. Yao, "Memetic search for vehicle routing with simultaneous pickup-delivery and time windows," *Swarm Evol. Comput.*, vol. 66, Oct. 2021, Art. no. 100927.
- [14] L. Feng et al., "Towards faster vehicle routing by transferring knowledge from customer representation," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 2, pp. 952–965, Feb. 2022.
- [15] Y. Huang, L. Feng, M. Li, Y. Wang, Z. Zhu, and K. C. Tan, "Fast vehicle routing via knowledge transfer in a reproducing kernel Hilbert space," *IEEE Trans. Syst., Man, Cybern., Syst.*, vol. 53, no. 9, pp. 5404–5416, 2023.
- [16] K. Zhang, X. Lin, and M. Li, "Transformer-based reinforcement learning for pickup and delivery problems with late penalties," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 12, pp. 24649–24661, Aug. 2022.
- [17] J. Li, L. Xin, Z. Cao, A. Lim, W. Song, and J. Zhang, "Heterogeneous attentions for solving pickup and delivery problem via deep reinforcement learning," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 3, pp. 2306–2315, Mar. 2022.
- [18] A. G. Dragomir, T. Van Woensel, and K. F. Doerner, "The pickup and delivery problem with alternative locations and overlapping time windows," *Comput. Oper. Res.*, vol. 143, Jul. 2022, Art. no. 105758.
- [19] E. Osaba, X.-S. Yang, I. Fister, J. D. Ser, P. Lopez-Garcia, and A. J. Vazquez-Pardavila, "A discrete and improved bat algorithm for solving a medical goods distribution problem with pharmacological waste collection," *Swarm Evol. Comput.*, vol. 44, pp. 273–286, Feb. 2019.

- [20] K. Treleaven, M. Pavone, and E. Frazzoli, "Asymptotically optimal algorithms for one-to-one pickup and delivery problems with applications to transportation systems," *IEEE Trans. Autom. Control*, vol. 58, no. 9, pp. 2261–2276, Sep. 2013.
- [21] J. Cai, Q. Zhu, Q. Lin, L. Ma, J. Li, and Z. Ming, "A survey of dynamic pickup and delivery problems," *Neurocomputing*, vol. 554, Oct. 2023, Art. no. 126631.
- [22] H. N. Psaraftis, M. Wen, and C. A. Kontovas, "Dynamic vehicle routing problems: Three decades and counting," *Networks*, vol. 67, no. 1, pp. 3–31, Jan. 2016.
- [23] S. Wandelt, S. Wang, C. Zheng, and X. Sun, "AERIAL: A meta review and discussion of challenges toward unmanned aerial vehicle operations in logistics, mobility, and monitoring," *IEEE Trans. Intell. Transp. Syst.*, early access, 2023, doi: [10.1109/TITS.2023.3343713](https://doi.org/10.1109/TITS.2023.3343713).
- [24] M. Savelsbergh and M. Sol, "Drive: Dynamic routing of independent vehicles," *Oper. Res.*, vol. 46, no. 4, pp. 474–490, Aug. 1998.
- [25] M. R. Swihart and J. D. Papastavrou, "A stochastic and dynamic model for the single-vehicle pick-up and delivery problem," *Eur. J. Oper. Res.*, vol. 114, no. 3, pp. 447–464, May 1999.
- [26] A. M. Arslan, N. Agatz, L. Kroon, and R. Zuidwijk, "Crowd-sourced delivery—A dynamic pickup and delivery problem with ad hoc drivers," *Transp. Sci.*, vol. 53, no. 1, pp. 222–235, Feb. 2019.
- [27] S. Mitrović-Minić, R. Krishnamurti, and G. Laporte, "Double-horizon based heuristics for the dynamic pickup and delivery problem with time windows," *Transp. Res. B, Methodol.*, vol. 38, no. 8, pp. 669–685, Sep. 2004.
- [28] M. Gendreau, F. Guertin, J.-Y. Potvin, and R. Séguin, "Neighborhood search heuristics for a dynamic vehicle dispatching problem with pickups and deliveries," *Transp. Res. C, Emerg. Technol.*, vol. 14, no. 3, pp. 157–174, Jun. 2006.
- [29] Y. Ma et al., "A hierarchical reinforcement learning based optimization framework for large-scale dynamic pickup and delivery problems," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 34, 2021, pp. 23609–23620.
- [30] Z. Su, W. Li, J. Li, and B. Cheng, "Heterogeneous fleet vehicle scheduling problems for dynamic pickup and delivery problem with time windows in shared logistics platform: Formulation, instances and algorithms," *Int. J. Syst. Sci., Oper. Logistics*, vol. 9, no. 2, pp. 199–223, Apr. 2022.
- [31] J. Cai, Q. Zhu, and Q. Lin, "Variable neighborhood search for a new practical dynamic pickup and delivery problem," *Swarm Evol. Comput.*, vol. 75, Dec. 2022, Art. no. 101182.
- [32] V. Pureza and G. Laporte, "Waiting and buffering strategies for the dynamic pickup and delivery problem with time windows," *Inf. Syst. Oper. Res.*, vol. 46, no. 3, pp. 165–175, Aug. 2008.
- [33] S. Vonolfen and M. Affenzeller, "Distribution of waiting time for dynamic pickup and delivery problems," *Ann. Oper. Res.*, vol. 236, no. 2, pp. 359–382, Jan. 2016.
- [34] K. Fagerholt, B. A. Foss, and O. J. Horgen, "A decision support model for establishing an air taxi service: A case study," *J. Oper. Res. Soc.*, vol. 60, no. 9, pp. 1173–1182, Sep. 2009.
- [35] M. W. Ulmer, B. W. Thomas, A. M. Campbell, and N. Woyak, "The restaurant meal delivery problem: Dynamic pickup and delivery with deadlines and random ready times," *Transp. Sci.*, vol. 55, no. 1, pp. 75–100, Jan. 2021.
- [36] J. Du, Z. Zhang, X. Wang, and H. C. Lau, "A hierarchical optimization approach for dynamic pickup and delivery problem with LIFO constraints," *Transp. Res. E, Logistics Transp. Rev.*, vol. 175, Jul. 2023, Art. no. 103131.
- [37] X. Li et al., "Learning to optimize industry-scale dynamic pickup and delivery problems," in *Proc. IEEE 37th Int. Conf. Data Eng. (ICDE)*, Apr. 2021, pp. 2511–2522.
- [38] G. Ghiani, E. Manni, A. Quaranta, and C. Triki, "Anticipatory algorithms for same-day courier dispatching," *Transp. Res. E, Logistics Transp. Rev.*, vol. 45, no. 1, pp. 96–106, 2009.
- [39] D. Sáez, C. E. Cortés, and A. Núñez, "Hybrid adaptive predictive control for the multi-vehicle dynamic pick-up and delivery problem based on genetic algorithms and fuzzy clustering," *Comput. Oper. Res.*, vol. 35, no. 11, pp. 3412–3438, Nov. 2008.
- [40] G. Ghiani, A. Manni, and E. Manni, "A scalable anticipatory policy for the dynamic pickup and delivery problem," *Comput. Oper. Res.*, vol. 147, Jun. 2022, Art. no. 105943.
- [41] M. Schilde, K. F. Doerner, and R. F. Hartl, "Metaheuristics for the dynamic stochastic dial-a-ride problem with expected return transports," *Comput. Oper. Res.*, vol. 38, no. 12, pp. 1719–1730, Dec. 2011.
- [42] M. Schilde, K. F. Doerner, and R. F. Hartl, "Integrating stochastic time-dependent travel speed in solution methods for the dynamic dial-a-ride problem," *Eur. J. Oper. Res.*, vol. 238, no. 1, pp. 18–30, Oct. 2014.
- [43] X. Xu and Z. Wei, "Dynamic pickup and delivery problem with transshipments and LIFO constraints," *Comput. Ind. Eng.*, vol. 175, Jan. 2023, Art. no. 108835.
- [44] J. Hao et al., "Introduction to the dynamic pickup and delivery problem benchmark - ICAPS 2021 competition," 2022, *arXiv:2202.01256*.
- [45] S. Liu, N. Lu, W. Hong, C. Qian, and K. Tang, "Effective and imperceptible adversarial textual attack via multi-objectivization," 2021, *arXiv:2111.01528*.
- [46] Y. Meliani, Y. Hani, S. L. Elhaq, and A. E. Mhamedi, "A Tabu search based approach for the heterogeneous fleet vehicle routing problem with three-dimensional loading constraints," *Appl. Soft Comput.*, vol. 126, Sep. 2022, Art. no. 109239.
- [47] Y. Tian, T. Zhang, J. Xiao, X. Zhang, and Y. Jin, "A coevolutionary framework for constrained multiobjective optimization problems," *IEEE Trans. Evol. Comput.*, vol. 25, no. 1, pp. 102–116, Feb. 2021.
- [48] J. Li, J. Cai, T. Sun, Q. Zhu, and Q. Lin, "Multitask-based evolutionary optimization for vehicle routing problems in autonomous transportation," *IEEE Trans. Autom. Sci. Eng.*, 2023, doi: [10.1109/TASE.2023.3326315](https://doi.org/10.1109/TASE.2023.3326315).
- [49] H. Li and Q. Zhang, "Multiobjective optimization problems with complicated Pareto sets, MOEA/D and NSGA-II," *IEEE Trans. Evol. Comput.*, vol. 13, no. 2, pp. 284–302, Apr. 2009.
- [50] S. Liu, J. Li, Q. Lin, Y. Tian, and K. C. Tan, "Learning to accelerate evolutionary search for large-scale multiobjective optimization," *IEEE Trans. Evol. Comput.*, vol. 27, no. 1, pp. 67–81, Feb. 2023.
- [51] Y. Su, Q. Lin, Z. Ming, and K. C. Tan, "Adapting decomposed directions for evolutionary multiobjective optimization," *IEEE Trans. Cybern.*, vol. 53, no. 10, pp. 6289–6302, 2023.
- [52] S. Liu, Q. Lin, K. C. Tan, M. Gong, and C. A. C. Coelho, "A fuzzy decomposition-based multi/many-objective evolutionary algorithm," *IEEE Trans. Cybern.*, vol. 52, no. 5, pp. 3495–3509, May 2022.
- [53] F. Glover, "Future paths for integer programming and links to artificial intelligence," *Comput. Oper. Res.*, vol. 13, no. 5, pp. 533–549, Jan. 1986.
- [54] F. Glover, "Tabu search—Part II," *ORSA J. Comput.*, vol. 2, no. 1, pp. 4–32, Feb. 1990.
- [55] Y. Huang, Y. Jin, and K. Hao, "Decision-making and multi-objectivization for cost sensitive robust optimization over time," *Knowl.-Based Syst.*, vol. 199, Jul. 2020, Art. no. 105857.
- [56] J. D. Knowles, R. A. Watson, and D. W. Corne, "Reducing local optima in single-objective problems by multi-objectivization," in *Proc. 1st Int. Conf. Evol. Multi-Criterion Optim.* Cham, Switzerland: Springer, 2001, pp. 269–283.
- [57] X. Ma, Z. Huang, X. Li, Y. Qi, L. Wang, and Z. Zhu, "Multiobjectivization of single-objective optimization in evolutionary computation: A survey," *IEEE Trans. Cybern.*, vol. 53, no. 6, pp. 3702–3715, Jun. 2023.
- [58] J. Shi, J. Sun, and Q. Zhang, "Multi-objectivization inspired metaheuristics for the sum-of-the-parts combinatorial optimization problems," *Appl. Soft Comput.*, vol. 103, May 2021, Art. no. 107157.
- [59] M. R. Ferrández, J. L. Redondo, B. Ivorra, Á. M. Ramos, and P. M. Ortigosa, "Preference-based multi-objectivization applied to decision support for high-pressure thermal processes in food treatment," *Appl. Soft Comput.*, vol. 79, pp. 326–340, Jun. 2019.
- [60] D. F. Lochtefeld and F. W. Ciarallo, "Multiobjectivization via helper-objectives with the tunable objectives problem," *IEEE Trans. Evol. Comput.*, vol. 16, no. 3, pp. 373–390, Jun. 2012.
- [61] D. F. Lochtefeld and F. W. Ciarallo, "Multi-objectivization via decomposition: An analysis of helper-objectives and complete decomposition," *Eur. J. Oper. Res.*, vol. 243, no. 2, pp. 395–404, Jun. 2015.
- [62] F. Carrabs, J.-F. Cordeau, and G. Laporte, "Variable neighborhood search for the pickup and delivery traveling salesman problem with LIFO loading," *INFORMS J. Comput.*, vol. 19, no. 4, pp. 618–632, Nov. 2007.
- [63] J. Alcalá-Fdez et al., "KEEL: A software tool to assess evolutionary algorithms for data mining problems," *Soft Comput.*, vol. 13, no. 3, pp. 307–318, Feb. 2009.
- [64] H.-F. Wang and Y.-Y. Chen, "A genetic algorithm for the simultaneous delivery and pickup problems with time window," *Comput. Ind. Eng.*, vol. 62, no. 1, pp. 84–95, Feb. 2012.